

RESPONSE TO REVIEWS (PAPER 509 → 1515)

Dear Meta Reviewer and Referees:

We have substantially revised the paper following the Referees' suggestions. For the convenience of Referees, changes to the paper are color-coded in blue. We would like to thank the Referees for insightful comments and for supporting this work!

Below please find our responses to the reviews.

Response to the revision items raised by the Meta Reviewer.

- (1) Clarify the novelty of OpenMind relative to RecLogic and TIEs. Explicitly explain how OMRs differ from the TIE prediction rules introduced in [28]. Identify which components are inherited, which are extended, and which are fundamentally new. [28] should be introduced early in the introduction, and the paper's incremental contributions over that work should be stated transparently.
- (2) Provide a complete description of the OMR discovery algorithm. Expand the current rule-discovery discussion into a technically complete account of how graph patterns and candidate OMRs are generated from user-item interaction graphs. Specify the search space, candidate-generation procedure, pruning criteria, scoring or selection method, computational complexity, and scalability mechanisms. The paper should contain enough detail for readers to understand and reproduce this central component.
- (3) Establish the quality, reliability, and diversity of discovered OMRs. Explain why rules mined from user-item interaction graphs should represent valid and meaningful viewpoint relationships. Report quantitative or qualitative evidence concerning rule precision, stability, coverage, and sensitivity to noisy or sparse interactions. Analyze whether the discovered rules are themselves sufficiently diverse and whether they generalize beyond the training data.
- (4) Validate the filter-bubble metrics against human judgment. The current evaluation relies heavily on automated measures such as random-walk-based FBP and ILD, which do not directly demonstrate that users perceive a filter bubble as being reduced. Conduct a validation study showing whether these metrics correlate with human assessments of viewpoint exposure and filter-bubble mitigation. Clearly justify the random-walk-with-restart formulation and analyze its robustness to parameter choices. Provide complete information about participant recruitment.
- (5) Broaden the empirical validation of model-agnostic integration. Evaluate OpenMind with multiple substantially different recommender systems rather than only RecLogic. This is necessary to support the claim that OpenMind is a plug-and-play, model-agnostic layer. Explain what information and interfaces OpenMind requires from a recommender, whether TIEs or equivalent rules are mandatory, and what engineering or modeling barriers arise when deploying it with a new recommendation system.
- (6) Improve organization, notation, examples, and database relevance. Reorganize the introduction so that the relationship to [28], the exact research gap, and the paper's database-oriented contributions are evident from the beginning. Strengthen the discussion of data-management aspects to better establish PVLDB relevance. Define terms such as TIE before first use, standardize notation, and substantially revise Example 2 and Figure 2.

[A] Many thanks! We have addressed point (1) in the response to comments R2[D1] and R3[W2, D2, C2]; point (2) in the response to R2[D3] and R3[W3, D4, C3]; point (3) in the response to R3[W3, D4, D7, C3]; point (4) in the response to R6[W3] and R2[D5]; point (5) in the response to R2[D2] and R3[C4]; and point (6) in the response to R3[W1, W2, D1, D2, D3, D6, C1, C2] and R6[W1, W2].

We have substantially rewritten the paper to accommodate the newly added clarifications, algorithmic details, and experiments.

Response to the comments of Referee #2.

[W1] *The novelty of the proposed OpenMind framework over existing diversification approaches could be better justified.*

[A] Thanks! We have further compared OpenMind against existing diversification work in Section 7 (p. 12) along three dimensions. (1) *Viewpoint-level diversity*. Unlike existing recommendation diversification methods [12, 14, 22–24, 68, 103], which mainly diversify topics or items within a fixed candidate set, OpenMind derives a same-topic alternative of a different or opposing viewpoint. (2) *Individual-level diversification*. Unlike population-level propagation methods [43, 94], OpenMind generates a personalized complementary set $I_d(x)$ for each user x at query time. (3) *Rule-based generation*. OpenMind uses OMRs to jointly encode topical relatedness, viewpoint difference, relevance, and engagement, and evaluates anchors and alternatives through coupled query-time inference rather than post-hoc retrieve-then-diversify processing.

[W2 & D5] *The experimental evaluation could be further strengthened, particularly by examining whether the proposed framework effectively mitigates filter bubbles in practice.*

The paper is motivated by the important societal issue of mitigating filter bubbles. However, the experimental evaluation mainly focuses on recommendation diversity metrics, e.g., ILD and FBP, which do not directly measure whether filter bubbles are actually mitigated or whether users are exposed to more diverse viewpoints in practice.

[A] Thanks! We have extended Exp-6 with two new user studies.

(1) *Top-5 user study*. Participants evaluated full top-5 lists. Among 46 valid responses, Figure 9(m) shows that OpenMind improves view divergence (ViewD) by 13.05% (resp. 4.35%) over SameTopic (resp. RAPID). Although Random attains the highest ViewD, its Acc, TopCon and All fall below OpenMind by 34.79%, 50.00% and 23.91%, indicating that Random includes items users are unlikely to accept (p. 11). Together with top-1 study (Figure 9(l)), this confirms that OpenMind provides diverse yet acceptable perspectives.

(2) *Human validation of metrics*. In addition, 58 annotators rated 40 lists on viewpoint diversity and exposure beyond prior beliefs (p. 11). In Figure 9(n), ILD and FBP correlate with human ratings by 0.76 and 0.74, respectively, with the statistical significance $p < 0.001$ of each correlation (i.e., the observed correlations are extremely unlikely to occur by random chance), validating both metrics.

[W3 & D6] *The design choices of the proposed OpenMind framework could be better justified.*

The design choices in Sections 4 and 5 could be better justified. While the paper presents efficient algorithms for CTR and TopK recommendation, it does not discuss the underlying design space or explain why the proposed designs are preferred over other plausible alternatives.

[A] Thanks! We have revised Sections 4 and 5 to compare plausible alternatives and justify our choices, with ablations in Section 6.

(1) *CTR evaluation*. A sequential design derives anchors with TIEs and then evaluates OMRs, repeating overlapping pattern matches; independent per-rule maps likewise forfeit reuse (p. 3). We instead group compatible TIEs and OMRs and refine a shared candidate map to a fixpoint (pp. 4-5). This preserves the sequential result while being up to $13.7\times$ faster than Seq (Figure 8(a), Table 1, and p. 9).

(2) *Ranking score*. Ranking only by dissimilarity may surface unacceptable content, while ranking only by acceptability yields insufficient diversity. We thus combine both through tradeoff parameter λ (p. 4). Figure 9(a) shows that the two endpoint designs optimize only one objective, whereas intermediate values achieve substantially better accuracy-diversity balance, supporting their joint use (p. 9).

(3) *TopK processing*. One may materialize all anchor-alternative pairs and sort them; per-item checking and full scoring repeat rule evaluation and score candidates that cannot enter the answer (p. 6). TopK instead uses weakly coupled stages, batched validation, and safe upper-bound pruning to focus on top- k anchors and avoid repeated TIE evaluation and full scoring (pp. 6-8). Against variants removing these designs, TopK is up to $1.46\times$, $1.87\times$ and $1.89\times$ faster than TopK_{na} , TopK_{nb} and TopK_{np} (Figure 9(k)), respectively (p. 10).

[D1] *The paper presents OMRs as its first contribution. However, OMRs appear to be highly similar to the prediction rules (TIEs) proposed in the existing work RecLogic. Both are logic rules augmented with ML predicates and share nearly identical formulations. The key difference seems to be that an OMR jointly models an anchor item and an alternative item, whereas a TIE only models a single recommended item. Given this close similarity, the paper should better justify the novelty of OMRs and clarify why they constitute a fundamentally new rule formalism rather than a straightforward extension of TIEs.*

[A] Thanks! We have clarified the relationship between OpenMind and RecLogic at two levels: Section 1 summarizes their differences in query semantics, rule representation and evaluation (p. 1), and Section 2.2 adds a rule-level comparison of OMRs and TIEs (p. 3).

(1) *Inherited: rule-guided recommendation*. OpenMind inherits the general paradigm of RecLogic: embedding ML predicates into logical graph rules and evaluating them over an enriched user-item interaction graph. Both TIEs and OMRs therefore combine graph patterns, logical conditions, and recommender scores (p. 1).

(2) *New: different query semantics*. A TIE answers whether a prediction $\text{Rec}(x, y)$ should be retained. An OMR instead answers an anchor-conditioned query: given user x and anchor y , it derives an alternative w that is acceptable to x , topically related to y , yet viewpoint-different from y . OMRs derive relationships among (x, y, w) rather than refine an individual user-item prediction (p. 1).

(3) *New: rule representation*. This new semantics requires general graph patterns over ternary (x, y, w) and heterogeneous logical and ML predicates for recommendation, dissimilarity, and popularity. In contrast, TIEs use restricted binary dual-star patterns and attribute conditions to refine a single recommendation predicate (p. 1&3).

(4) *New: different evaluation problem*. TIEs can be evaluated independently for each user-item pair. In contrast, OMRs must preserve user-anchor-alternative dependencies, reuse matches across TIEs

and OMRs, and rank anchor-alternative pairs. This yields the coupled CTR and exact TopK problems absent from RecLogic (p. 1).

Thus, OpenMind inherits RecLogic’s rule-guided paradigm while introducing new query semantics, rule representations, and evaluation algorithms for viewpoint-level diversification.

[D2] *The paper claims model-agnostic integration as its second contribution, stating that OpenMind can serve as a plug-and-play framework for arbitrary recommender systems. However, this claim is not yet fully supported. On the one hand, the proposed algorithms (e.g., Figure 5) explicitly rely on TIEs defined in the downstream recommender RecLogic, suggesting a tighter coupling with the underlying recommendation model. On the other hand, the experimental evaluation is conducted using only a single recommender. Evaluating OpenMind on a broader range of recommendation models would provide stronger evidence for its claimed model generality.*

[A] Thanks! We have clarified both the interface and the empirical evidence for model-independent integration (pp. 3&11).

(1) *Minimal interface*. OpenMind requires no RecLogic-specific TIEs. Any base score $\mathcal{M}_{\text{rec}}(x, y)$ can be exposed via the degenerate TIE

$$Q[x_0, y_0](\mathcal{M}_{\text{rec}}(x_0, y_0) \rightarrow \text{Rec}(x_0, y_0)),$$

where Q contains only x_0 and y_0 , and the rule has no additional logical conditions X_1 . Our CTR and TopK algorithms then use this rule to identify anchors and OMRs to derive alternatives. No access to the model architecture, parameters, or training procedure is required. Deployment additionally requires the enriched graph and pluggable item-level signals $\mathcal{M}_{\text{dis}}$ and sim_T used by the OMRs.

(2) *Empirical generality*. We have further evaluated OpenMind with BPRMF, LighterX and ConvMF beyond RecLogic. Across these distinct recommenders, OpenMind improves ILD by 71.05% and FBP by 57.48% on average (Figure 9(o)), while maintaining comparable accuracy. Its query time remains 47–219 ms (p. 11), showing that the same CTR apply without recommender-specific modifications.

These results confirm that OpenMind is a model-independent query-time layer: RecLogic is one supported instantiation, while a standard ML recommender is covered as its special single-TIE case.

[D3] *The rule discovery component is not fully described. Although Figure 3 includes a rule discovery module, the paper does not explain how Open Mind Rules (OMRs) are actually discovered. In particular, OMRs are defined over graph patterns, but the paper does not specify how these patterns are generated or identified from large-scale user-item graphs. A more detailed description of the rule discovery process, including its algorithmic design and scalability, would significantly improve the completeness of the proposed framework.*

[A] Thanks! We have added Section 3.2 (p. 4), which specifies the discovery input/output, search space, candidate and predicate generation, scoring support/confidence pruning, and scalability. Full pseudo code and complexity analysis are given in [4]. Moreover, we have added Exp-9 to further show that extending pre-mined TIEs accelerates OMR discovery and scales steadily with graph size (p. 12). Discovery is adapted from TIE learning; our main contribution remains scalable query-time evaluation of the resulting rules.

[D4] *The paper distinguishes viewpoint diversification from conventional topic-level diversification in its motivation. However, it remains unclear how this distinction is reflected in the OMR formulation.*

OMRs identify alternative items based primarily on dissimilarity and acceptability scores. It is not well explained how these criteria ensure that the selected item expresses an alternative viewpoint on the same topic, rather than simply being another diverse item.

[A] Thanks! We have clarified (p. 3-4) that OMRs achieve same-topic viewpoint diversification in three complementary ways.

(1) *Structural topical alignment*. The graph pattern Q explicitly connects anchor y_0 and alternative w_0 through shared context, e.g., a common group or category. Hence, only candidates with verifiable graph-level evidence of topical relatedness are considered (p. 3).

(2) *Viewpoint-distinguishing predicates*. The precondition X includes predicates encoding contrasting opinions or attributes, together with $M_{\text{dis}}(y_0, w_0)$. Thus, the alternative can differ from the anchor in viewpoint, rather than merely in topic (p. 3). See Example 2.

(3) *Ranking refinement*. The score $s_{\text{dis}}(u, v, w)$ combines content dissimilarity and topic proximity $\text{sim}_T(v, w)$ (p. 4), preventing unrelated but dissimilar items from ranking highly. It balances s_{dis} against user acceptability s_{acc} (together with M_{aggr} in X).

Our ablation (Figure 9(j)) confirms that SameTopic, which preserves topic overlap but removes viewpoint predicates, yields much lower ILD than OpenMind (p. 10). Thus, topical relatedness alone is insufficient and the viewpoint-level predicates are essential.

Many thanks for your support and helpful comments!

Response to the comments of Referee #3.

[W1] *The presentation of the paper could be better.*

[A] Thanks! We have substantially revised the presentation by (1) reorganizing Section 1 to introduce RecLogic [28] and TIEs before OpenMind and clarify their differences; (2) revising Figures 1-2 and Example 2 to label variables and explain same-topic viewpoint diversification; and (3) expanding Section 3 to specify the requirements, deployment barriers, and complete OMR-discovery workflow, including its search space, generation, pruning and scalability.

[W2 & D2 & C2] *Previous work [28] could be cited before page 3.*

I would recommend citing Reference [28] earlier in the paper to avoid giving readers the impression, during the first three pages, that the proposed approach is entirely novel.

Reorganise the introduction to make room for ref [28].

[A] Thanks! We have rewritten Section 1 to introduce RecLogic and TIEs [28] before OpenMind (p. 1). We now state that OpenMind inherits the general use of ML predicates in logical graph rules from RecLogic. It differs in (1) *query semantics*: TIEs refine $\text{Rec}(x, y)$, while OMRs derive an alternative w conditioned on user x and anchor y ; (2) *rules*: OMRs use general graph patterns and heterogeneous predicates; and (3) *evaluation*: OMRs require coupled anchor-alternative inference and pairwise ranking, leading to our new CTR and TopK algorithms (please also see our response to R2[D1]).

[W3 & D4 & C3 & D5] *OMRs mining from user-item interaction graphs is almost not discussed, while central in OpenMind.*

Could you explain why OMRs are discovered from the user-item interaction graph? How reliable are these rules? What evidence supports their validity? This point is never discussed, even in the experiments.

Page 4, Rule discovery: the description is too short to be useful.

Justify the quality and the reliability of the OMRs discovered from the user-item interaction graph.

[A] Thanks! We have made two changes as suggested.

(1) *Discovery rationale and algorithm*. We have added Section 3.2 (p. 4), which explains why OMRs are mined from enriched user-item graphs and presents the discovery input/output, search space, candidate generation, pruning and scalability. By combining interactions with content, knowledge and social relations, the graph exposes recurring structures linking anchors to topically related, viewpoint-different and acceptable alternatives. Only rules meeting the support and confidence thresholds are retained; full algorithm and complexity analysis are given in [4]. Exp-9 (p. 12) further shows that extending pre-mined TIEs makes OMR discovery consistently faster than TIE discovery (see also our response to R2[D3]).

(2) *Quality and reliability*. Exp-8 (p. 11) evaluates coverage, held-out generalization, semantic precision, robustness to noisy and sparse interactions, discovery stability, and rule diversity. The mined OMRs cover 73.06% of test user-item pairs, change performance by less than 7.97% on unseen data, achieve 0.7836 annotator-judged precision, and remain robust and stable under noise, sparsity, and random training splits. The OMRs also exhibit structural, predicate, and answer diversity, confirming that the rules are complementary.

[W4 & D8] *The real life case study is quite interesting but not discussed with enough detail.*

Exp-6: User Study, page 12: please give more detail about the people implied in this evaluation, their background ... Could you clarify how the participants were selected for the evaluation?

[A] Thanks! We have expanded Exp-6 (user study, p. 11).

(1) *Recruitment and background*. We have clarified that we recruited 90 participants, aged 18-44, from several universities and research institutes, including graduate students, research staff, and software engineers in computer science. All were familiar with online recommendation scenarios and filter bubbles.

(2) *Quality control*. We used cross-checking questions for quality control. For each metric, participants selected both the best and the best two recommendations. Responses were discarded if the latter did not include the former; 76 of 84 (resp. 46 of 60) responses remained valid for top-1 (resp. top-5) study.

(3) *Procedure*. Participants received each target user's interaction history, together with item descriptions and topic labels, enabling them to judge whether an alternative differs in viewpoint from the anchor while remaining acceptable to the user.

(4) *Future work*. We have added real-world deployment and online A/B testing of OpenMind as future work (p. 12).

[D1 & C1] *Example 2, page 3, (1): I get lost with variables x_3 and x_6 . To be clarified. Example 2, which refers to Figure 2, is difficult to follow because the node-related structures (e.g., $L(x_0)$) are not provided. Consequently, the reader must guess the associated attributes to understand the example.*

Example 2, which refers to Figure 2, has to be clarified.

[A] As suggested, we have now labeled every variable and edge in Figures 1 and 2 (p. 2), stated the semantic role of each variable in Example 2, and traced how the pattern and predicates jointly

establish same-topic viewpoint difference (p. 3).

[D3] Page 3, 1st column: "attributes A and B exist at vertices $h(x)$ and $h(y)$ " => You could use the notation $F_A(h(x))$ here, isn't it?

[A] Thanks! We have used $A \in F_A(h(x))$ and $B \in F_A(h(y))$ (p. 2).

[D6] First occurrence of TIE occurs before its definition (page 3).

[A] Thanks! Since TIEs now first appear on p. 1, we define them there as logical rules that determine whether a user-item prediction should be retained. The formal syntax is reviewed in Section 2 (p. 3).

[D7] Are the discovered OMRs themselves diverse? (page 9)

[A] Thanks! We have added an experiment evaluating OMR diversity through structural, predicate-set, and answer-set dissimilarity. We compare size-matched high-support Σ_{OMR}^{sup} and random subset Σ_{OMR}^{rand} , and separately report the coverage of the full set Σ_{OMR} . Exp-8 (p. 11) shows that Σ_{OMR}^{sup} is internally less diverse than Σ_{OMR}^{rand} , i.e., high-support OMRs capture dominant patterns. Diversity emerges at the level of the full set: Σ_{OMR} attains higher coverage than Σ_{OMR}^{sup} and keeps covering new test pairs when additional OMRs are added. Thus, these lower-support rules are complementary rather than redundant, and Σ_{OMR} as a whole is diverse.

[C4] Explain the barriers to deploying OpenMind on a recommender?

[A] Thanks! As explained in R2[D2], OpenMind can directly support standard recommenders through our CTR and TopK algorithms, without RecLogic-specific TIEs or model retraining. Our experiments with three additional recommenders further show consistent diversity gains with comparable accuracy, confirming that integration requires no algorithmic changes (p. 11).

The main barrier arises when item-level signals are unavailable: $\mathcal{M}_{dis}$ needs evidence of viewpoint difference, while sim_T needs evidence of topical relatedness. Inferring these signals from raw content requires additional annotation or external models (p. 3).

Many thanks for your support and constructive comments!

Response to the comments of Referee #6.

[W1] This paper is not relevant to PVLDB. It focuses purely on the question of the filter bubble, not the management of the data.

[A] As suggested, we have revised Section 1 (p. 1) to make the database contribution explicit. Filter-bubble mitigation is the motivating application, and *query-time viewpoint diversification* is the database problem: given a recommended anchor, return a relevant alternative expressing a different view on the same topic. The core technical problem is *scalable rule-based query evaluation over large user-item interaction graphs*. Evaluating OMRs requires graph-pattern matching, logical and ML-predicate evaluation, and ranked answer generation under strict query-time constraints.

OpenMind addresses three core database problems. (1) *Scalable graph-rule evaluation*: OMRs combine general graph patterns with heterogeneous logical and ML predicates; our algorithms avoid exhaustive rule-by-rule evaluation via shared matches, batching, and safe pruning. (2) *Multi-query optimization*: CTR jointly evaluates interdependent rule sets over a shared candidate map, reusing intermediate matches and avoiding redundant graph-pattern matching [33, 47, 84]. (3) *Top-k processing*: TopK uses batched validation

and score upper bounds to prune candidates that cannot enter the answer, without materializing all anchor-alternative pairs [27, 55].

Thus, OpenMind formulates query-time viewpoint diversification and solves its key data-management issues via scalable graph-rule evaluation, multi-query optimization, and top- k processing.

[W2] Only <10% of citations are relevant from PVLDB, SIGMOD, etc., another indication that this paper is not relevant to data management.

[A] Thanks! We have revised Section 7 to better organize the related work around the three data-management foundations of OpenMind: graph-rule evaluation (including subgraph and pattern matching), multi-query optimization and exact top- k query processing (p. 12). The revised bibliography includes 40 papers from PVLDB, SIGMOD, ICDE, VLDBJ, TKDE or TODS, which cover rule-based graph processing [29, 31–34, 36, 38, 39], subgraph/pattern matching [10, 11, 35, 37, 48, 61, 79, 89, 91, 107, 111], multi-query optimization [80, 82, 84, 98], and top- k optimization [21, 105, 109]. In addition, we retain complementary data-mining and recommendation work because filter-bubble mitigation is inherently interdisciplinary.

[W3] Even ignoring the relevancy concerns, the core problem this paper tackles is filter bubbles. But the paper also uses a brittle metric of random walk w/ restart. I am not convinced the paper tackles the core problem it aims to. In particular, the method the authors use to determine if a filter bubble is broken is via an automated random walk metric. There is no investigation if this corresponds to human judgement, especially in the context of the current technique. This is critical given the paper's claims.

[A] Thanks! We have added three studies to address this concern.

(1) *RWR is not inherent to OpenMind*. RWR is neither our definition of a filter bubble nor an evaluation metric; it is one instantiation of the pluggable predicate $\mathcal{M}_{dis}(y, w)$. This predicate may instead use random-walk-, distance-, or learning-based models [15, 52, 57, 58, 62]. We adopt RWR because filter bubbles often manifest as community separation in interaction graphs (pp. 8–9).

To test dependence on this choice, we have also instantiated $\mathcal{M}_{dis}$ with Jaccard distance, cosine similarity and SimCLR [15]. Figure 9(p) shows that OpenMind consistently outperforms RAPID in ILD (resp. FBP) across all instantiations by 27.86% (resp. 44.27%) (p. 11).

(2) *Validation via human judgment*. RWR is used only to instantiate $\mathcal{M}_{dis}$; ILD and FBP are computed independently. We recruited 58 annotators to rate 40 recommendation lists from OpenMind on perceived viewpoint diversity and exposure beyond prior beliefs. Figure 9(n) shows that ILD and FBP correlate with human judgments by 0.76 and 0.74, respectively, with the statistical significance $p < 0.001$ of each correlation; hence they reflect users' perceived reduction of filter bubbles (p. 11; see R2[W2&D5]).

(3) *Robustness to RWR parameters*. We varied the restart probability γ from 0.1 to 0.6. Figure 9(q) shows that FBP vary by less than 0.49%. This stability arises because thresholding $\mathcal{M}_{dis}(y_0, w_0)$ absorbs moderate RWR-score fluctuations, and moreover, the other conditions in each OMR reduce sensitivity to any single parameter (p. 11).

Together, these results show that OpenMind is neither tied to RWR nor dependent on brittle parameter choices, and that its evaluation metrics are consistent with human perceptions.

Many thanks for helping us improve the paper!

OpenMind: Query-Time Diversification via Logical Rules

Wenfei Fan^{1,2,3}, Kehan Pang³, Dandan Lin¹

¹Shenzhen Institute of Computing Sciences ²University of Edinburgh ³Beihang University
{fanwenfei, lindandan}@sics.ac.cn, pangkehan@buaa.edu.cn

ABSTRACT

Recommender systems can reinforce filter bubbles by repeatedly exposing users to views aligned with their prior beliefs. We present OpenMind, a lightweight query-time framework that augments recommendations with relevant items offering alternative or opposing viewpoints, without retraining the underlying model. OpenMind uses logical Open Mind Rules (OMRs) over user-item interaction graphs to relate each recommended anchor to acceptable, viewpoint-diverse alternatives. We develop efficient algorithms for coupled OMR evaluation in both CTR and top- k query processing, sharing inference with the base recommender and avoiding exhaustive candidate materialization. Using real-world datasets, we empirically verify that OpenMind improves exposure to diverse views at negligible query processing cost without loss in accuracy.

PVLDB Artifact Availability: The source code, data, and/or other artifacts have been made available at <https://github.com/hdqlkncso24jvf/OpenMind>.

1 INTRODUCTION

When Frank and Nancy watch the same debate video but hold opposing views, the recommender system may subsequently show Frank only comments supporting his position and Nancy only comments supporting hers. Although both engage with the same topic, each is exposed to only one side of the debate, reinforcing their prior beliefs and potentially deepening disagreement. This illustrates a *filter bubble* (a.k.a. echo chamber, information cocoon) [44, 74, 76]. Such bubbles are reinforced by modern machine-learning (ML) based recommender systems, which learn from users’ historical interactions (e.g., clicked or watched) over a *user-item interaction graph* [65, 104, 108] and favor content aligned with prior views.

Breaking filter bubbles is critical: they hinder creativity and innovation [71, 73], undermine democratic processes [20, 92], and amplify manipulation and misinformation [97]. Meanwhile, users seek diverse viewpoints to understand the spectrum of opinions, reassess their beliefs, and construct counterarguments [8, 45].

Motivated by this need, we study *query-time viewpoint diversification*: given an item already recommended to a user, find a relevant alternative expressing a different or opposing view on the same topic. Existing approaches are insufficient. Re-ranking methods [12, 14, 22, 23, 68, 78, 103] mainly promote topic-level diversity and operate on a pre-retrieved candidate set already biased by the user’s history, where the desired viewpoint may be absent. Influence-propagation methods [9, 42, 43, 53, 72, 94] optimize the global diffusion of opposing content, but do not answer how to serve alternative viewpoints to individual users at query time. Thus, existing methods do not answer how to generate personalized, same-topic alternative viewpoints during recommendation.

A natural direction is rule-guided recommendation. RecLogic [28] embeds ML predicates into logical rules called TIEs (predicate rules). A TIE decides whether a user-item prediction should be retained, but cannot answer the anchor-conditioned query central

to viewpoint diversification: given a recommended item, which acceptable alternative presents a different view on the same topic?

OpenMind. We propose OpenMind, a lightweight query-time layer for *viewpoint-level diversification*. Given a set $I(x)$ of anchor items returned by a base recommender for user x , OpenMind produces a complementary set $I_d(x)$ of relevant and engaging alternatives that express different or opposing views on the same topic. These alternatives are returned as optional extended answers, preserving the original recommendations and requiring no model retraining.

At the core of OpenMind are *Open Mind Rules* (OMRs), a class of rules which combine graph patterns, logical conditions, and ML predicates. An OMR relates a user, an anchor and candidate alternatives, and combines structural context with recommendation, dissimilarity and acceptability conditions to determine when an alternative should be returned. OpenMind requires only a user-item interaction graph (optionally enriched with knowledge graphs or social networks) and a scoring interface from the base recommender; it does not require TIEs or model retraining.

OpenMind inherits from RecLogic only the general paradigm of embedding ML predicates in logical graph rules [28]; its query and execution problems are different. (1) *Query semantics*: a TIE refines a binary prediction $\text{Rec}(x, y)$, whereas an OMR derives an alternative w conditioned jointly on user x and anchor y . (2) *Rule representation*: this anchor-conditioned semantics requires general graph patterns over ternary pivots (x, y, w) and heterogeneous conditions that jointly enforce relevance, topical alignment, viewpoint difference, and popularity. (3) *Evaluation*: anchors and alternatives cannot be evaluated independently; OMRs require shared inference across interdependent rules and ranking over anchor-alternative pairs.

Database problem and technical challenges. Although filter-bubble mitigation motivates OpenMind, realizing it at scale requires rule-based query evaluation over large user-item graphs under strict latency constraints. This raises three core database problems. (1) *Scalable graph-rule evaluation*: OMRs require matching general graph patterns and checking heterogeneous logical and ML predicates. (2) *Multi-query optimization*: evaluating TIEs and OMRs sequentially repeats overlapping pattern matches; CTR (Click-Through Rate) instead shares intermediate matches across interdependent rules [33, 47, 84]. (3) *Exact top- k processing*: materializing all anchor-alternative pairs is wasteful; TopK uses batched validation and score bounds to prune candidates without sacrificing exactness [27, 55].

Contributions & Organization. Our contributions are as follows.

(1) *Anchor-conditioned viewpoint diversification* (Section 2). We introduce OMRs to relate a user x , a recommended anchor y , and an alternative w that offers a different viewpoint on the same topic while remaining relevant and acceptable. Unlike TIEs, which refine a single user-item prediction, OMRs define ternary, anchor-conditioned query semantics over (x, y, w) .

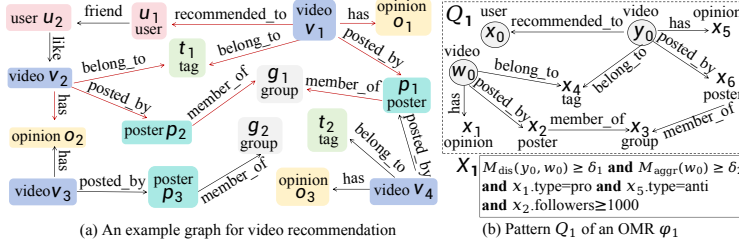


Figure 1: A graph and a pattern for video recommendation

(2) *Model-independent integration and rule discovery* (Section 3). We develop OpenMind that requires only a user-item interaction graph and a recommendation-scoring interface, without retraining the base model. OpenMind discovers OMRs from graph structure and ML signals, and applies them to produce complementary alternatives $I_d(x)$ alongside the original anchors $I(x)$.

(3) *Scalable rule-based query processing* (Sections 4 and 5). We develop CTR (Click-Through Rate) and TopK algorithms for coupled rule evaluation and exact top- k answering. CTR shares intermediate graph matches across interdependent rules, avoiding the pipeline of first deriving anchors and then alternatives. TopK further combines batched validation with safe score bounds to prune candidates without materializing all anchor-alternative pairs. We prove the coupled execution sound and complete *w.r.t.* the sequential baseline.

(4) *Empirical results* (Section 6). Using real-world datasets and multiple recommenders, we show that OpenMind substantially improves viewpoint diversity while preserving accuracy and efficiency. (a) For CTR, while retaining comparable accuracy, OpenMind improves the diversity metrics ILD and FBP (state-of-the-art) methods by 35.62% and 46.06% on average, respectively, up to 110.28% and 144.30%. Top- k improves diversity $ILD@k$ and $FBP@k$ by 42.19% and 45.98%, respectively. (b) Answering a CTR query for a user takes 0.12s on average on graphs with 225M edges, 13.7 $\times$ faster than the sequential pipeline. For top- k , it takes 86ms on average, up to 16.4 $\times$ faster. (c) *User studies confirm a perceived reduction of filter bubbles mitigated by OpenMind. Both ILD and FBP correlate with human judgments by 0.76 and 0.74.* (d) *OpenMind is model-agnostic, improving ILD and FBP by 71.05% and 57.48% on average across all base recommenders while preserving accuracy and query time.* (e) *We validate the quality, robustness and diversity of OMRs.*

Related and future work are in Sections 7 and 8; proofs are in [4].

2 RULES FOR KEEPING AN OPEN MIND

This section presents OMRs, Open Mind Rules. We first review basic notations (Section 2.1), and then introduce OMRs (Section 2.2).

2.1 Preliminaries

We start with notations. Assume a countably infinite set Λ of labels.

Graphs. Consider directed labeled graphs $G = (V, E, L, F_A)$, where (a) V is a finite set of vertices; (b) $E \subseteq V \times \Lambda \times V$ is a finite set of labeled edges, where (v, l, v') denotes an edge labeled $l \in \Lambda$ from v to v' ; and (c) each vertex $v \in V$ has a label $L(v) \in \Lambda$ and a finite tuple of attributes $F_A(v) = (A_1 = a_1, \dots, A_n = a_n)$, written as $v.A_i = a_i$. Vertices may have different attributes without a fixed schema.

Patterns. A *graph pattern* is $Q[\bar{x}, x_0, y_0, w_0] = (V_Q, E_Q, L_Q)$, where (1) V_Q (resp. E_Q) is a finite set of *pattern vertices* (resp. *edges*); (2) L_Q assigns each pattern vertex $v \in V_Q$ a label $L_Q(v) \in \Lambda$; (3) $\bar{x}$ is

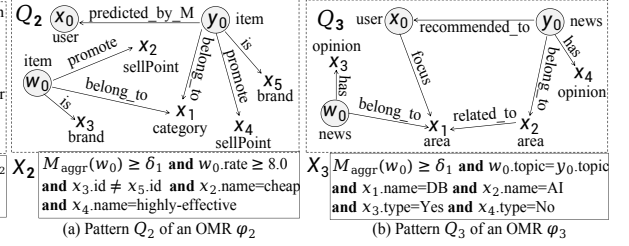


Figure 2: Two OMRs φ_2 and φ_3

a list of distinct variables, each denoting a vertex v in V_Q ; and (4) among $\bar{x}$, x_0 denotes a user, y_0 an *anchor* item, and w_0 a candidate *alternative*. We call (x_0, y_0, w_0) the designated variables of Q .

Pattern matching. A *match* of pattern $Q[\bar{x}, x_0, y_0, w_0]$ in graph G is defined as a homomorphism h from Q to G such that (a) for each pattern vertex $v \in V_Q$, $L_Q(v) = L(h(v))$; and (b) for each pattern edge $e = (v, l, v')$ in Q , $e' = (h(v), l, h(v'))$ is an edge in G . We denote the match by the vector $h(\bar{x})$, consisting of $h(x)$ for all $x \in \bar{x}$.

Example 1: Figure 1(a) shows a labeled video-recommendation graph G , and Figure 1(b) shows a pattern Q_1 with designated variables (x_0, y_0, w_0) . A match of Q_1 in G is highlighted in red. $\square$

2.2 Open Mind Rules

We next define OMRs, starting with their predicates.

Predicates. A *predicate* over pattern $Q[\bar{x}, x_0, y_0, w_0]$ has the form:

$$x.A \oplus y.B \mid x.A \oplus c \mid \mathcal{M}(x, y) \otimes \delta \mid \mathcal{M}(y) \otimes \delta \mid \text{Rec}(x, y),$$

where $\oplus$ is one of $=, \neq, <, \leq, >, \geq$; $\otimes$ is one of $<, \leq, >, \geq$; x and y are variables in $\bar{x}$; c is a constant; A and B are attributes in Υ ; $\text{Rec}(x, y)$ is a predicate that is true when item y is recommended to user x , and $\mathcal{M}(x, y)$ is an ML predicate, including but not limited to:

- A recommendation model $\mathcal{M}_{\text{rec}}(x, y)$ outputs a score in $[0, 1]$ indicating user preference, e.g., NCF [51], DeepFM [46], BPRMF [81], LightGCN [49], LighterX [114], and ConvMF [59].
- A model $\mathcal{M}_{\text{dis}}(y, w)$ that measures item dissimilarity via classical distance metrics (e.g., cosine similarity [62], Hamming distance [58]), random walk-based metrics (e.g., RWR [66], SimRank [57]), or learning methods (e.g., CML [52], SimCLR [15]).
- A model $\mathcal{M}_{\text{aggr}}(y)$ that estimates item popularity using click/rating/exposure frequency [110, 115], power-law modeling [19], regression-based methods [7, 85], or RL-based approaches [86].

All prediction scores produced by $\mathcal{M}$ are normalized to $[0, 1]$ and compared against a predefined threshold $\delta \in [0, 1]$.

OMRs. An OMR (Open Mind Rule) φ has the form:

$$Q[x_0, y_0, w_0](X \rightarrow \text{Rec}_d(x_0, w_0)),$$

where $Q[x_0, y_0, w_0]$ is a graph pattern; $\text{Rec}_d(x_0, w_0)$ is true when item w_0 is recommended to user x_0 as an alternative to y_0 ; and X is a conjunction of predicates over $Q[x_0, y_0, w_0]$ that include at least one predicate of either the form $\text{Rec}(x_0, y_0)$ or $\mathcal{M}_{\text{rec}}(x_0, y_0) \otimes \delta$.

We refer to Q and $X \rightarrow \text{Rec}_d(x_0, w_0)$ as the *pattern* and *dependency*, and X and $\text{Rec}_d(x_0, w_0)$ as the *precondition* and *consequence* of φ , respectively. We consider OMRs that pertain to the same recommendation model $\mathcal{M}_{\text{rec}}$, referred to as OMRs *pertaining to* $\mathcal{M}_{\text{rec}}$.

Semantics. Denote by h a match of the pattern Q of an OMR φ in graph G , and by p a predicate in its precondition X . We say that h satisfies p , written $h \models p$, if (a) for $p = x.A \oplus y.B$, **attributes** $A \in F_A(h(x))$ and $B \in F_A(h(y))$, and $h(x).A \oplus h(y).B$ holds; similarly

for $x.A \oplus c$; (b) for $p = \mathcal{M}_{\text{rec}}(x, y) \otimes \delta$, the score $\mathcal{M}_{\text{rec}}(h(x), h(y))$ satisfies $\otimes \delta$; likewise for $\mathcal{M}_{\text{dis}}(y, w)$ and $\mathcal{M}_{\text{aggr}}(w)$; and (c) for $p = \text{Rec}(x, y)$, a possibly hidden Rec-edge exists from $h(x)$ to $h(y)$.

We write $h \models X$ if $h \models p$ for every $p \in X$. For $\varphi = Q[x_0, y_0, w_0](X \rightarrow \text{Rec}_d(x_0, w_0))$, each match h satisfying X derives $h(w_0)$ as an alternative to anchor $h(y_0)$ for user $h(x_0)$.

Comparison with TIEs. A TIE has the form $Q[x_0, y_0](\mathcal{M}_{\text{rec}}(x_0, y_0) \wedge X_1 \rightarrow \text{Rec}(x_0, y_0))$ [28], where Q is a pair of disjoint stars centered at x_0 and y_0 : each star contains paths sharing only their pivot. The precondition X_1 contains attribute predicates $z_1.A \oplus z_2.B$ and $z.A \oplus c$.

In contrast, an OMR uses a general pattern over (x_0, y_0, w_0) and may combine an anchor condition with $\mathcal{M}_{\text{dis}}(y_0, w_0)$, $\mathcal{M}_{\text{aggr}}(w_0)$, and attribute predicates. Thus, syntactically, TIEs use restricted binary dual-star patterns, whereas OMRs use general ternary patterns and heterogeneous predicates. Semantically, a TIE validates one user-item prediction, whereas an OMR derives an alternative conditioned on an anchor. As noted in Section 1, this further requires coupled anchor-alternative evaluation and pairwise ranking.

Enforcing viewpoint diversification. An OMR identifies same-topic alternative viewpoints in three complementary ways.

(1) *Structural topical alignment.* The pattern Q connects anchor y_0 and alternative w_0 through shared topical context, such as a common tag, topic or category. Hence, only candidates with graph-level evidence of topical relatedness are considered.

(2) *Viewpoint-distinguishing predicates.* The precondition X may encode contrasting opinions, stances or attributes, with $\mathcal{M}_{\text{dis}}(y_0, w_0)$, distinguishing viewpoint difference from generic item difference.

(3) *Ranking refinement.* We will define the score $s_{\text{dis}}(u, v, w)$ that weights content dissimilarity by topic proximity $\text{sim}_T(v, w)$, preventing unrelated items from ranking highly. It is balanced against user acceptability score $s_{\text{acc}}(u, w)$ with $\mathcal{M}_{\text{aggr}}$ in X (see Section 4).

Example 2: Figures 1(b) and 2 show three example OMRs.

(1) OMR φ_1 is $Q_1[x_0, y_0, w_0](\mathcal{M}_{\text{rec}}(x_0, y_0) \wedge X_1 \rightarrow \text{Rec}_d(x_0, w_0))$, where X_1 is $\mathcal{M}_{\text{dis}}(y_0, w_0) \geq \delta_1 \wedge \mathcal{M}_{\text{aggr}}(w_0) \geq \delta_2 \wedge x_2.\text{followers} \geq 1000 \wedge x_1.\text{type} = \text{pro} \wedge x_5.\text{type} = \text{anti}$. In Q_1 , x_0 is a user; y_0, w_0 are videos; x_1, x_5 their opinions; x_2, x_6 their posters; and x_3 the posters' shared group. The rule suggests alternative video w_0 to user x_0 when $\mathcal{M}_{\text{rec}}$ recommends anchor y_0 , both videos share topic tag x_4 but express opposing opinions (by $\mathcal{M}_{\text{dis}}$), and w_0 is popular (by $\mathcal{M}_{\text{aggr}}$) and posted by a credible user (i.e., acceptable). Thus, φ_1 returns an opposite-viewpoint alternative while preserving engagement.

(2) Figure 2(a) shows $\varphi_2 = Q_2[x_0, y_0, w_0](\mathcal{M}_{\text{rec}}(x_0, y_0) \geq \delta \wedge X_2 \rightarrow \text{Rec}_d(x_0, w_0))$, where X_2 is $\mathcal{M}_{\text{aggr}}(w_0) \geq \delta_1 \wedge w_0.\text{rate} \geq 8.0 \wedge x_3.\text{id} \neq x_5.\text{id} \wedge x_2.\text{name} = \text{cheap} \wedge x_4.\text{name} = \text{highly-effective}$. In Q_2 , x_0 is a user; y_0, w_0 are items; x_1 is their shared category; x_2, x_4 are their selling points; and x_3, x_5 are their brands. The rule suggests alternative w_0 when $\mathcal{M}_{\text{rec}}$ recommends anchor y_0 , both items share category x_1 but differ in brand and selling point, and w_0 is popular and highly rated. Thus, φ_2 returns a popular, same-category alternative with contrasting product characteristics.

(3) Figure 2(b) shows $\varphi_3 = Q_3[x_0, y_0, w_0](\text{Rec}(x_0, y_0) \wedge X_3 \rightarrow \text{Rec}_d(x_0, w_0))$, where X_3 is $x_1.\text{name} = \text{DB} \wedge x_2.\text{name} = \text{AI} \wedge \mathcal{M}_{\text{aggr}}(w_0) \geq \delta_1 \wedge y_0.\text{topic} = w_0.\text{topic} \wedge x_3.\text{type} = \text{Yes} \wedge x_4.\text{type} = \text{No}$. In Q_3 , x_0 is a user; y_0, w_0 are news items; x_1, x_2 are related topic

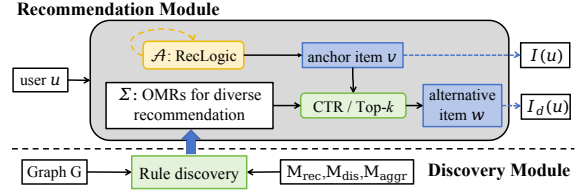


Figure 3: Recommendation framework OpenMind

areas; and x_3, x_4 are their opinions. The rule suggests w_0 when anchor y_0 is recommended, x_0 follows a related topic, w_0 is popular, and the two news items express different opinions. Thus, φ_3 returns a topically related alternative with a different perspective. $\square$

3 MAKING RECOMMENDERS OPEN-MINDED

This section presents OpenMind, our recommendation framework.

3.1 Model-Independent Integration

Overview. As shown in Figure 3, OpenMind operates as a light-weight query-time layer over a base recommender $\mathcal{A}$. Given an enriched user-item graph G , it discovers a set Σ_{OMR} of OMRs offline. At query time, $\mathcal{A}$ produces an anchor set $I(u)$ for user u , and OpenMind applies Σ_{OMR} to generate a complementary set $I_d(u)$ of relevant items expressing alternative or opposing viewpoints. The labeled alternatives are presented alongside the anchors for voluntary exploration, without modifying or retraining $\mathcal{A}$.

OpenMind supports CTR and top- k recommendation. A sequential approach first derives anchors and then evaluates OMRs for their alternatives, repeating overlapping graph matches. Instead, CTR jointly derives anchors and alternatives in one inference process, while TopK further uses ranking and pruning to return the highest-scoring anchor-alternative pairs (Sections 4 and 5).

Minimal interface. OpenMind requires only (a) a user-item interaction graph G , optionally enriched with content, knowledge, or social relations, and (b) a scoring interface $\mathcal{M}_{\text{rec}}(u, v)$ from the base recommender. It does not require access to the recommender's architecture, parameters, or training procedure.

Any recommender $\mathcal{M}_{\text{rec}}$ can be represented by a single TIE

$$Q[x_0, y_0](\mathcal{M}_{\text{rec}}(x_0, y_0) \rightarrow \text{Rec}(x_0, y_0)),$$

where Q contains only user x_0 and item y_0 , and the TIE has no additional logical precondition. Hence, conventional recommenders are special cases of RecLogic and can be processed directly by our CTR and TopK algorithms: this TIE identifies anchors, and discovered OMRs derive their alternatives by using CTR or TopK.

Beyond $\mathcal{M}_{\text{rec}}$, OpenMind uses two item-level signals: $\mathcal{M}_{\text{dis}}(v, w)$ assesses viewpoint difference, and $\text{sim}_T(v, w)$ measures topical relatedness. When suitable topic labels, stance annotations, or content embeddings are unavailable, these signals must be inferred from raw content using external classifiers or language models.

In this paper we instantiate OpenMind over RecLogic [28] because both represent recommendations as logical graph rules, allowing TIEs and OMRs to be discovered and evaluated within one framework. Moreover, RecLogic already refines uncertain ML predictions with logical conditions, making alternative generation over its outputs more challenging than over raw model scores. This is nevertheless an implementation choice: any recommender can be

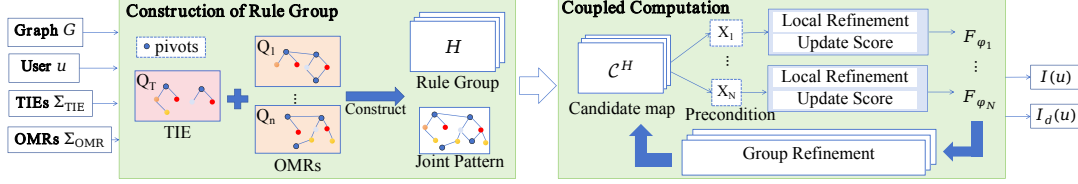


Figure 4: Overview of coupled CTR algorithm.

handled by using the degenerate-TIE representation above.

3.2 OMR Discovery

OMR discovery is a one-time offline step adapted from the TIE-discovery method of RecLogic [28]; the full algorithm and complexity analysis are given in the appendix [4].

Discovery problem. We enrich the user-item graph G with content, knowledge, and social relations that provide evidence of user preferences, topical relatedness, viewpoint difference, and item quality.

Given G , training data T , pre-mined TIEs Σ_{TIE} , models $\mathcal{M}_{\text{rec}}$, $\mathcal{M}_{\text{dis}}$ and $\mathcal{M}_{\text{aggr}}$, and thresholds (r_1, r_2, α) , our discovery algorithm, OMR Learner, returns OMRs with support at least r_1 , confidence at least r_2 , and at most α variables. Support counts covered positive examples, while confidence is their fraction among all covered examples. We use α to balance expressivity and scalability.

Search space. For each TIE φ_T with (Q_T, X_T) , we search for OMRs φ with (Q, X) , where pattern $Q = Q_T \cup \rho_B \cup Q_w$. Here path ρ_B connects anchor y_0 to a new item variable w_0 , and Q_w is a star centered at w_0 . The precondition X extends X_T with predicates over the new variables and constraints on $\mathcal{M}_{\text{dis}}(y_0, w_0)$ and $\mathcal{M}_{\text{aggr}}(w_0)$.

Candidate generation. For each TIE φ_T with (Q_T, X_T) , we build the type-level schema S of G , identify schema paths from y_0 to an item variable w_0 as bridges ρ_B , and combine them with stars Q_w centered at w_0 . Each $Q_T \cup \rho_B \cup Q_w$ within the α -variable bound forms a candidate pattern. For each candidate, it derives empirical distributions of $\mathcal{M}_{\text{dis}}(y_0, w_0)$ and $\mathcal{M}_{\text{aggr}}(w_0)$ from matches satisfying X_T , selects representative quantile thresholds, and levelwise specializes the resulting base OMR with applicable predicates over ρ_B and Q_w .

Scoring, pruning, and scalability. Each candidate is scored by support, measuring its coverage of positive training examples, and confidence, measuring the precision of its predictions. A candidate satisfying both r_1 and r_2 is returned; one satisfying r_1 but not r_2 is further specialized. Although discovery is exponential in the worst case, we control the search by bounding pattern size, generating schema-level extensions, selecting representative thresholds, and pruning any infrequent candidate together with its refinements, whose support cannot increase as predicates are added (see [4]). Discovery is performed offline and incurs no query-time overhead.

4 CTR RECOMMENDATION

This section develops CTR, the algorithm of OpenMind for click-through-rate recommendations enriched with diverse viewpoints. We formalize the CTR recommendation problem as follows.

- *Input:* A graph G , a traditional recommender RecLogic with a set Σ_{TIE} of TIEs, a user u and a set Σ_{OMR} of OMRs
- *Output:* A set $I(u)$ of anchors by Σ_{TIE} and a set $I_d(u)$ of highest-scoring alternatives by Σ_{OMR} such that for each item $w \in I_d(u)$, there exists at least one match h of an OMR φ in G such that $h \models \text{Rec}_d(u, w)$ by referencing an anchor $v \in I(u)$.

Design space and challenges. Three natural designs expose distinct limitations. (1) *Sequential evaluation.* Deriving anchors with TIEs and then evaluating OMRs for each anchor repeatedly matches overlapping patterns; even deciding whether a general OMR pattern has a match is NP-hard [41]. (2) *Independent evaluation.* Evaluating each rule with its own candidate map duplicates overlapping intermediate states across compatible rules. (3) *Full materialization.* An OMR may produce many alternatives for the same user-anchor pair; materializing them all is costly and may retain weakly relevant, insufficiently distinct, or unacceptable items.

Our design. We thus make three choices. First, we couple anchor and alternative inference rather than use a sequential pipeline. Second, compatible TIEs and OMRs share one candidate map, allowing overlapping matches to be reused. Third, we incrementally rank alternatives by both viewpoint dissimilarity and user acceptability instead of materializing all matches. Section 6 compares the coupled design with the sequential baseline and ablates the ranking tradeoff.

Since the ranking score determines the highest-scoring alternatives returned by CTR, we first define it in Section 4.1, and then present the coupled-fixpoint CTR algorithm in Section 4.2.

4.1 Ranking Score

Given a user-anchor pair (u, v) , we aim to select an alternative item w that presents a different viewpoint from v without reducing user satisfaction. Dissimilarity alone may surface extreme but unacceptable content, increasing user churn; acceptability alone largely reproduces the original filter bubble. We therefore define a ranking score $s(u, v, w)$ that joins dissimilarity and acceptability:

$$s(u, v, w) = \lambda \cdot s_{\text{dis}}(u, v, w) + (1 - \lambda) \cdot s_{\text{acc}}(u, w),$$

where $s_{\text{dis}}(u, v, w)$ measures viewpoint dissimilarity, $s_{\text{acc}}(u, w)$ captures user acceptability, and $\lambda \in [0, 1]$ controls the tradeoff. A higher score indicates a more promising alternative with respect to (u, v) . Note that $\lambda = 1$ and $\lambda = 0$ recover the two degenerate designs. Section 6 will vary λ to validate this tradeoff.

Next, we elaborate the dissimilarity score and acceptability score.

Dissimilarity score. The dissimilarity score $s_{\text{dis}}(u, v, w)$ quantifies how well item w serves as an alternative to anchor v for user u . An effective alternative should remain topically related to v while presenting sufficiently different content. Thus, we define $s_{\text{dis}}(u, v, w)$ by combining topic proximity and content dissimilarity. Since different OMRs capture distinct structural and feature-based signals, their individual contributions are also incorporated into the score:

$$s_{\text{dis}}(u, v, w) = \frac{1}{|\Sigma_{\text{OMR}}|} \cdot \text{sim}_T(v, w) \cdot \sum_{\varphi \in \Sigma_{\text{OMR}}^w} s_{\text{dis}}(u, v, w, \varphi),$$

where $\text{sim}_T(v, w)$ is the topic proximity of item w w.r.t. item v , Σ_{OMR}^w is the set of OMRs that have at least one match h in G at (u, v) such that $h(w_0) = w$, and $s_{\text{dis}}(u, v, w, \varphi) \in [0, 1]$ is the individual content dissimilarity of w w.r.t. v under an OMR φ in Σ_{OMR}^w . The dissimilarity term combines viewpoint difference with topical proximity, so unrelated but distant items do not receive high scores.

Input: $G(V, E)$, u , a set Σ_{OMR} of OMRs, a set Σ_{TIE} of TIEs, $\mathcal{M}_{\text{dis}}$, $\mathcal{M}_{\text{aggr}}$ and λ .

Output: a set $I(u)$ of anchor items and a set $I_d(u)$ of alternative items.

1. $A_v := \emptyset$ for each item vertex $v \in V$; $I(u) := \emptyset$; $I_d(u) := \emptyset$;
2. **for** each φ_T in Σ_{TIE} **do**
3. $H, Q^H, Q_D^H := \text{constructGroup}(u, \varphi_T, \Sigma_{\text{OMR}})$;
4. $C^H := \text{InitCand}(Q^H, u)$; $F_H := C^H$;
5. **while** F_H does not converge **do**
6. **for** each $\varphi \in H$ **do**
7. $F_\varphi := \text{localRefinement}(F_H, Q_D^H, \varphi)$;
8. **for** each pair $(v, w) \in F_\varphi(y_0, w_0)$ **do**
9. $s(u, v, w) := \text{updateScore}(u, v, w, \mathcal{M}_{\text{dis}}, \mathcal{M}_{\text{aggr}}, \lambda)$;
10. $A_v := A_v \cup \{(w, s(u, v, w))\}$;
11. $F_H := \text{groupRefinement}(\{F_\varphi | \varphi \in H\})$;
12. $I(u) := I(u) \cup F_H(y_0)$;
13. $I_d(u) := I_d(u) \cup F_H(w_0)$ with the highest ranking score;
14. **Return** $I(u)$ and $I_d(u)$;

Figure 5: CTR algorithm.

The individual dissimilarity $s_{\text{dis}}(u, v, w, \varphi)$ under each individual OMR $\varphi \in \Sigma_{\text{OMR}}^w$ is formally defined as follows:

$$s_{\text{dis}}(u, v, w, \varphi) = \mathcal{M}_{\text{dis}}(v, w) \cdot \Delta_\varphi(u, v, w),$$

where $\Delta_\varphi(u, v, w)$ is the contribution of dissimilarity given by φ in terms of topology and vertex features (see more details in [4]).

Let $\gamma(v, w) = \text{sim}_T(v, w) \cdot \mathcal{M}_{\text{dis}}(v, w)$. Then, given a set Σ_{OMR} of OMRs, the content dissimilarity score of w becomes:

$$s_{\text{dis}}(u, v, w, \Sigma_{\text{OMR}}) = \frac{1}{|\Sigma_{\text{OMR}}|} \cdot \gamma(v, w) \cdot \sum_{\varphi \in \Sigma_{\text{OMR}}^w} \Delta_\varphi(u, v, w).$$

Acceptability score. To mitigate churn from opposing viewpoints, we estimate a user’s likelihood of engaging with an alternative item. Let $\mathcal{L}(u, v, \Sigma_{\text{OMR}})$ denote the alternatives at (u, v) generated by OMRs in Σ_{OMR} . For each $w \in \mathcal{L}(u, v, \Sigma_{\text{OMR}})$, we define

$$s_{\text{acc}}(u, w) = \mathcal{M}(u, w) \cdot \mathcal{M}_{\text{aggr}}(w),$$

where $\mathcal{M}(u, w) \in [0, 1]$ is the predicted probability that user u engages with w , and $\mathcal{M}_{\text{aggr}}(w)$ measures its global popularity. Intuitively, w is more acceptable if it aligns with user u ’s interests and is broadly endorsed, reducing abrupt exposure to opposing views.

Score-based CTR problem. For each anchor $v \in I(u)$, CTR returns the candidate $w \in \mathcal{L}(u, v, \Sigma_{\text{OMR}})$ maximizing $s(u, v, w)$.

4.2 Coupled CTR Algorithm

We next present our coupled CTR algorithm for the score-based CTR recommendation problem, with performance guarantees.

Overview. As Figure 4 shows, CTR processes each TIE in two steps.

(1) *Rule-group construction.* For each TIE $\varphi_T = Q_T[x_0, y_0](X_T \rightarrow \text{Rec}(x_0, y_0)) \in \Sigma_{\text{TIE}}$, we construct a group $\mathcal{H}(\varphi_T)$ of OMRs with compatible anchor-side constraints. **Grouping balances two extremes: independent maps forfeit reuse, whereas one global map induces an unnecessarily large joint pattern.** Each group thus shares a candidate map, reusing overlapping TIE-OMR matches.

(2) *Coupled refinement.* The shared map is iteratively updated by local and group refinement. **Local refinement enforces one OMR’s pattern and predicates; group one retains candidates supported by at least one rule under existential semantics.** The former preserves rule-specific correctness, while the latter enables cross-rule reuse without requiring all rules to hold. Refinement stops at a fixpoint.

Algorithm. We now present algorithm CTR in Figure 5. Given

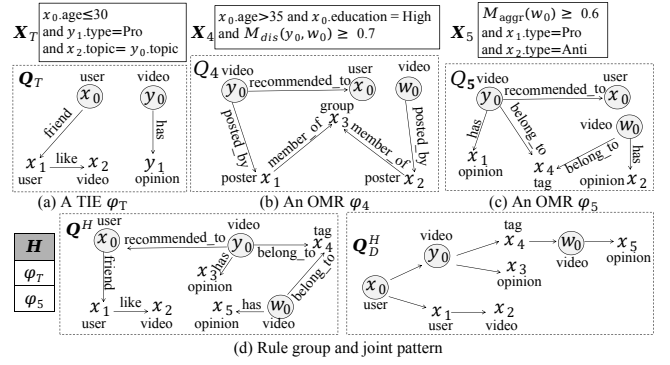


Figure 6: A running example of CTR

a graph $G(V, E)$, a user u , a set Σ_{OMR} of OMRs, a set Σ_{TIE} of TIEs, $\mathcal{M}_{\text{dis}}$, $\mathcal{M}_{\text{aggr}}$ and λ , CTR returns a set $I(u)$ of anchors and a set $I_d(u)$ of alternatives by computing the matches for TIEs and OMRs in a single yet shared candidate map via a coupled fixpoint procedure.

Algorithm CTR processes each TIE φ_T by constructing its rule group H , joint pattern Q^H , rooted DAG Q_D^H , and shared candidate map C^H (Lines 2–4). It then alternates local refinement, incremental score updates, and group refinement until convergence (Lines 5–11). Finally, once a fixpoint is reached, it extracts the surviving anchors and their highest-scoring alternatives (Lines 12–13).

Example 3: We illustrate CTR for the user u_1 on the graph in Figure 1(a). Suppose that Σ_{TIE} has one TIE φ_T , and Σ_{OMR} has two OMRs φ_4 and φ_5 , as shown in Figures 6(a), (b) and (c), respectively.

It first builds a rule group H containing φ_T and φ_5 , where φ_4 is excluded because its constraint on $x_0.\text{age}$ conflicts with that in φ_T . It also induces a joint pattern Q^H and its rooted DAG Q_D^H as shown in Figure 6(d). Then, it performs a local refinement by enforcing φ_T ’s constraints, making $C^H(x_0) = \{u_1\}$, $C^H(y_0) = \{v_1\}$, $C^H(x_1) = \{u_2\}$, $C^H(x_2) = \{v_2\}$, $C^H(x_3) = \{o_1\}$, $C^H(x_4) = \{t_1\}$, $C^H(w_0) = \{v_2\}$ and $C^H(x_5) = \{o_2\}$. For (v_1, v_2) , it updates the score accordingly. Next, it performs a local refinement by φ_5 , making no changes and so the group refinement reaches a fixpoint. It then terminates. $\square$

The used procedures are presented as follows (see more in [4]).

Procedure constructGroup. Given a user u , a set Σ_{OMR} of OMRs and a TIE $\varphi_T = Q_T[x_0, y_0](X_T \rightarrow \text{Rec}(x_0, y_0)) \in \Sigma_{\text{TIE}}$, this procedure constructs (a) a *rule group* $H = \mathcal{H}(\varphi_T)$ tailored for φ_T ; and induces (b) a *joint pattern* Q^H for this group H encoding both TIE and OMRs topology; and (c) builds a *rooted DAG* Q_D^H of joint pattern Q^H via a BFS traversal from user variable x_0 , enabling directional refinement.

Procedure InitCand. It initializes a shared candidate map C^H for user u and group H . It sets $C^H(x_0) = \{u\}$. For each variable $z \neq x_0$, $C^H(z)$ contains vertices in V satisfying all unary predicates on z .

Procedure localRefinement. Given the current candidate map C^H and a rule $\varphi(Q, X) \in H$, this procedure refines C^H to F_φ by enforcing (a) topological consistency with Q and (b) the constraints in X . It performs a forward pass and a backward pass over the DAG Q_D^H .

Unlike prior dynamic-programming approaches [47], we do not iterate these passes for each rule. Instead, refinement is coordinated at the group level, allowing convergence over H as a whole.

Procedure updateScore. When multiple OMRs or their matches produce the same alternative w for anchor v , updateScore incrementally aggregates their contributions to $s(u, v, w)$. Since $s_{\text{acc}}(u, w)$ and $\text{sim}_T(v, w)$ are fixed, they are computed once; each subsequent

match adds only $\lambda\gamma\Delta_\varphi(u, v, w)$. Specifically, for $(v, w) \in F_\varphi(y_0, w_0)$, if $w \in A_v$, we increase $s(u, v, w)$ by this amount; otherwise, we initialize $s(u, v, w) = (1 - \lambda)s_{\text{acc}}(u, w) + \lambda\text{sim}_T(v, w) \frac{s_{\text{dis}}(u, v, w, \varphi)}{|\Sigma|}$. Correctness follows from Lemma 1 (see [4] for a proof).

Lemma 1: *Given user u and an anchor v , for any alternative item w at which at least one OMR is applicable, UpdateScore is correct to compute the ranking score $s(u, v, w)$ of w w.r.t. v .* $\square$

Procedure groupRefinement. Rather than requiring all rules to hold simultaneously, groupRefinement adopts *existential rule support*: a candidate vertex for variable $z \in Q^H$ is retained if it is supported by at least one rule in H via a valid local refinement. This matches the witness-based semantics of TIEs and OMRs. Formally, given local refinements F_φ for each $\varphi \in H$, the updated candidate map is

$$F_H = \bigcup_{\varphi \in H} F_\varphi(C^H).$$

Starting from C^H , the algorithm iteratively applies

$$F_H^{i+1} := \text{groupRefinement}(F_H^i)$$

until convergence. Rules in H may be applied in any order. Once F_H reaches a fixpoint, we extract anchors from $F_H^\infty(y_0)$ that satisfy the TIE precondition X_T . For each anchor v , we choose the highest-scoring alternatives from $F_H^\infty(w_0)$ supported by some rule $\varphi \in H$.

Termination & confluence. By Lemma 2, F_H decreases monotonically over a finite candidate lattice and thus converges to a unique fixpoint. By Theorems 3 and 4, this fixpoint yields the same result as sequential evaluation while avoiding redundant exploration.

Lemma 2: *For any group H and candidate maps C^H, C'^H such that $C'^H(z) \sqsubseteq C^H(z)$ for all z , we have $F_H(C'^H) \subseteq F_H(C^H)$. Hence the coupled iteration converges to a unique fixpoint F_H^∞ .* $\square$

Proof sketch: We first prove that both local refinement F_φ and group refinement F_H are monotone. Since candidate sets are finite and monotonically shrink until stabilization, the iteration converges in finitely many steps to a unique fixpoint. $\square$

Theorem 3: *For any user u , TIE φ_T and rule group H , let $\text{Seq}(u, H)$ be the set of anchor-alternative pairs (v, w) produced by the sequential algorithm, and $\text{Cpl}(u, H)$ be the set of pairs extracted from the fixpoint F_H^∞ of our coupled algorithm. Then $\text{Seq}(u, H) = \text{Cpl}(u, H)$.* $\square$

Proof sketch: For the soundness, we show that any $(v, w) \in \text{Cpl}(u, H)$ satisfies the same TIE and OMR conditions as in the sequential algorithm. We prove the completeness by showing that any $(v, w) \in \text{Seq}(u, H)$ admits compatible TIE and OMR witnesses whose union is preserved by the coupled fixpoint. $\square$

Theorem 4: *For any user u and rule group H , let $\mathcal{N}_{\text{Seq}}$ be the set of candidate pairs explored by the sequential CTR algorithm, and $\mathcal{N}_{\text{Cpl}}$ be those by the coupled algorithm. Then $|\mathcal{N}_{\text{Cpl}}| \leq |\mathcal{N}_{\text{Seq}}|$.* $\square$

Proof sketch: We show the stronger inclusion $\mathcal{N}_{\text{Cpl}} \subseteq \mathcal{N}_{\text{Seq}}$, indicating that the coupled algorithm only prunes candidates earlier than the sequential pipeline and does not introduce new ones. $\square$

Complexity analysis. The time complexity of CTR is $O(|\Sigma_{\text{TIE}}| \cdot (|\Sigma_{\text{OMR}}| + |Q^H| + t|H||Q|_{\max}|G|))$, where t is the number of group refinements to reach a fixpoint, and $|Q|_{\max}$ is the maximum pattern size among rules in H . For each TIE $\varphi_T \in \Sigma_{\text{TIE}}$: (a) constructing the rule group costs $O(|\Sigma_{\text{OMR}}|)$ by checking pivot compatibility; (b) initializing the candidate map requires $O(|Q^H|)$; and (c) iter-

ative group refinement costs $O(t|H||Q|_{\max}|G|)$, since each local refinement takes $O(|Q||G|)$ time following [28].

Remark. Coupled evaluation avoids repeated TIE-OMR matching, rule grouping bounds shared state, and incremental scoring avoids full materialization. Monotone refinement over the candidate lattice guarantees a unique fixpoint equivalent to sequential evaluation.

5 TOP-K RECOMMENDATION

This section develops TopK, the algorithm of OpenMind to make top- k recommendations along with suggested diverse viewpoints. The top- k recommendation problem is formulated as follows.

- *Input:* $G, \Sigma_{\text{TIE}}, \Sigma_{\text{OMR}}$ and u as in CTR, and an integer k .
- *Output:* a set $K(u)$ of top- k ranked anchors with the highest recommendation strength, and a set $K_d(u)$ of top- k alternative items with the highest aggregated ranking scores w.r.t. $K(u)$.

Here for an alternative item w , its aggregated ranking score w.r.t. the selected top- k anchors in $K(u)$ is defined as

$$s(u, w|K(u)) = \max_{v \in K(u)} s(u, v, w).$$

Design space and challenges. A brute-force solution runs CTR to materialize all anchor-alternative pairs, then sorts them by score, and returns the top- k answers. Although exact, it incurs three avoidable costs. (1) *Excessive candidate generation.* Full coupling evaluates alternatives for anchors that cannot enter $K(u)$, whereas complete separation forfeits reusable anchor-side results. (2) *Repeated anchor validation.* Validating anchors individually repeatedly evaluates the same TIEs and graph structures. (3) *Unnecessary alternative scoring.* Exact scoring evaluates candidates whose maximum achievable scores already fall below the current top- k threshold.

Our solution. We develop an exact TopK algorithm with three complementary designs. (1) It uses *weakly coupled* stages, first computing the top- k anchors $K(u)$ and then restricting alternative generation to $K(u)$, reducing the candidate space while reusing anchor-side results. (2) It validates anchor candidates in batches, sharing TIE evaluation and avoiding unnecessary candidates. (3) It derives safe candidate-level score upper bounds to prune alternatives before full scoring. Together, these designs avoid materializing all anchor-alternative pairs while preserving exact top- k answers. Section 6 evaluates each design through ablations.

Unlike CTR, which returns alternatives for every anchor, TopK needs them only for the top- k anchors. Full coupling would waste work on discarded anchors, while complete separation loses reuse. We thus finalize $K(u)$ first and restrict OMR evaluation to these anchors while reusing intermediate results across the two stages.

Section 5.1 computes top- k anchors in batches, and Section 5.2 prunes alternatives over the selected anchors by safe score bounds.

5.1 Top- k Anchor Selection

We begin with the ranking strategy for selecting the top- k anchors.

Anchor ranking strategy. We rank anchors using the *updated* recommendation strength that combines the base prediction $\mathcal{M}_{\text{rec}}$ with TIE-based refinement. We handle items in two cases w.r.t. a fuzzy area $[\delta_L, \delta_H]$: (a) If $\mathcal{M}_{\text{rec}}(u, v) \geq \delta_H$, item v is directly retained as an anchor candidate. (b) If $\mathcal{M}_{\text{rec}}(u, v) \in [\delta_L, \delta_H]$, we apply TIEs at (u, v) . If some $\phi \in \Sigma_{\text{TIE}}$ derives $\text{Rec}(u, v)$, the score of v is raised to δ_H ; otherwise it is reduced to δ_L . Anchors are then selected as

the top- k items under the updated strengths. When multiple items receive the same adjusted value, their relative order follows the original ranking of M_{rec} . Thus, ordering preservation is maintained.

Lemma 5: For any two items v_1 and v_2 with $M_{\text{rec}}(u, v_1) > M_{\text{rec}}(u, v_2)$, if both enter the top- k set $K(u)$, v_1 is ranked before v_2 . $\square$

Batch validation via TIE aggregation. Validating candidates individually repeatedly evaluates the same TIEs. We instead maintain a sliding window over the M_{rec} -sorted list H_{rec} and validate candidates in batches. In each iteration, we select a contiguous set V_{sel} of promising anchors, initially the top- k items, and aggregate the TIEs in Σ_{TIE} whose constraints on y_0 are compatible with at least one $v \in V_{\text{sel}}$, forming a unified set $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ (see [4] for details).

TopKAnchor algorithm. We run coupled refinement over this unified set $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ like in CTR, which simultaneously validates all anchors in V_{sel} . After processing $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$, we discard those in V_{sel} that remain unsupported, and slide the window forward to include the next high- M_{rec} items. This process repeats until k anchors are determined. For limited space, its pseudocode is given in [4].

Example 4: Assume graph G has five item vertices and $k = 2$. Initially, TopKAnchor sorts items as $H_{\text{rec}} = \{v_1, \dots, v_5\}$, and none has base strength above δ_H , thus batch validation starts. In the first round, the window $[1, 2]$ gives $V_{\text{sel}} = \{v_1, v_2\}$ and $\Sigma_{\text{TIE}}^s(V_{\text{sel}}) = \{\varphi_1, \varphi_3\}$. Procedure Refinement evaluates these rules with $C(x_0) = \{u\}$ and $C(y_0) = \{v_1, v_2\}$. Only v_1 is validated, i.e., $V_{\text{valid}} = \{v_1\}$. The algorithm updates $K(u)$ and advances the window to $i_b = 3$ and $i_e = 3$. The process continues until k anchors are found. $\square$

Complexity. TopKAnchor takes $O(|V_{\text{item}}| \log |V_{\text{item}}|)$ time to sort the items. For each batch V_{sel} , the refinement over $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ takes $O(|\Sigma_{\text{TIE}}^s(V_{\text{sel}})| |Q| |C_\varphi|_{\max})$ time, where $|C_\varphi|_{\max}$ is the maximum size of the candidate map C_φ over all φ . This is because each refinement removes at least one vertex from a candidate map C_φ . Hence the overall complexity of TopKAnchor is $O(|V_{\text{item}}| \log |V_{\text{item}}| + |\Sigma_{\text{TIE}}^s(V_{\text{sel}})| |Q| |C_\varphi|_{\max})$. When $C_\varphi(y_0)$ is restricted to items in V_{sel} , the candidate space gets much smaller, further improving efficiency.

5.2 Coupled Top- k Algorithm

We now present TopK to identify the top- k anchors and alternatives.

Overview. Given a user u , TopK first identifies the top- k anchors $K(u)$ using TopKAnchor. It then finds a global top- k alternative set $K_d(u)$ induced by the selected anchors in $K(u)$, by applying the same local and group refinements as CTR over the OMRs in Σ_{OMR} .

To avoid local refinement for each OMR $\varphi \in \Sigma_{\text{OMR}}$ at each top- k anchor v , TopK uses early pruning based on a *candidate-level upper bound*. It maintains (a) a global max-heap Ψ of the current top- k alternatives and (b) a threshold τ , the k th-largest aggregated score in Ψ . Before refining a group H , TopK computes an upper bound $\hat{s}(u, w)$ for each candidate w . If $\hat{s}(u, w) < \tau$, then w cannot enter the top- k set and is pruned; otherwise, TopK applies local refinement and updates its score. As in CTR, group refinement progressively shrinks the candidate map and tightens the bounds until a fixpoint.

Below we define the upper bound, followed by details of TopK.

Candidate-level upper bound. Exact scoring evaluates every applicable OMR for each alternative. We instead prune a candidate when a safe upper bound on its final score falls below the current

Input: $G, u, \Sigma_{\text{OMR}}, \Sigma_{\text{TIE}}, M_{\text{rec}}, M_{\text{dis}}, M_{\text{aggr}}, \lambda, \delta_H$ and k .

Output: a set $K(u)$ of anchor items and a set $K_d(u)$ of alternative items.

1. $\Psi := \emptyset; K(u) := \emptyset; K_d(u) := \emptyset;$
2. $K(u) := \text{TopKAnchor}(u, \Sigma_{\text{TIE}}, M_{\text{rec}}, \delta_H, k);$
3. $H, Q^H, Q_D^H := \text{constructGroup}(u, K(u), \Sigma_{\text{OMR}});$
4. $C^H := \text{InitCand}(Q^H, u); F_H := C^H;$
5. Initialize best_w and $\hat{s}(u, w)$ for each $w \in F_H(w_0);$
6. **for** each $(v, w) \in F_H(y_0, w_0)$ **do**
7. Initialize $\hat{s}(u, v, w)$ by the current rule-level upper bounds;
8. **if** $\hat{s}(u, v, w) \geq \hat{s}(u, w)$ **then** $\hat{s}(u, w) := \hat{s}(u, v, w);$
9. **while** F_H does not converge **do**
10. **for** each $\varphi \in H$ **do**
11. $F_\varphi := \text{localRefinement}(F_H, Q_D^H, \varphi);$
12. **for** each $w \in F_\varphi(w_0)$ **do**
13. $\tau :=$ the k -th highest aggregated score in $\Psi;$
14. **if** $\hat{s}(u, w) < \tau$ **then** Remove (w) from $F_\varphi(w_0);$ **continue** ;
15. **for** each v such that $v \in F_\varphi(y_0)$ and $(v, w) \in F_\varphi(y_0, w_0)$ **do**
16. $s(u, v, w) := \text{updateScore}(u, v, w, M_{\text{dis}}, M_{\text{aggr}}, \lambda);$
17. **if** $s(u, v, w) > \text{best}_w$ **then**
18. $\text{best}_w := s(u, v, w);$
19. **if** $\text{best}_w > \tau$ **then** Update Ψ and τ by $w;$
20. Update (tighter) $\text{UB}_\varphi(u, v)$ and affected $\hat{s}(u, w')$ based on $F_\varphi;$
21. $F_H := \text{groupRefinement}(\{F_\varphi | \varphi \in H\});$
22. $K_d(u) := \Psi;$
23. **Return** $K(u)$ and $K_d(u);$

Figure 7: TopK algorithm.

k th-best score, preserving exactness. Given user u , top- k anchors $K(u)$, and rule group H , we define for each candidate alternative w a global upper bound over all anchors in $K(u)$:

$$\hat{s}(u, w) = \max_{v \in K(u)} \hat{s}(u, v, w),$$

where $\hat{s}(u, v, w)$ is the upper bound w.r.t. an anchor v , defined as:

$$\hat{s}(u, v, w) = \lambda \cdot \frac{1}{|H|} \sum_{\varphi \in H} \text{UB}_\varphi(u, v) + (1 - \lambda) \cdot s_{\text{acc}}(u, w),$$

where $\text{UB}_\varphi(u, v)$ is a rule-level upper bound, defined as the maximum (possible) ranking score of an *arbitrary* alternative w found by φ at (u, v) pair. Formally, we define $\text{UB}_\varphi(u, v)$ by considering the increment contributed by φ to an arbitrary w w.r.t. (u, v) :

$$\text{UB}_\varphi(u, v) = \hat{y}(v) \cdot \Delta_\varphi^{\max},$$

where $\Delta_\varphi^{\max}$ denotes the maximum score increment contributed by an OMR φ , i.e., $\Delta_\varphi^{\max} = \max_{w \in F_\varphi(w_0|v)} \Delta_\varphi(u, v, w)$, and $\hat{y}(v) = \max_{w \in F_\varphi(w_0|v)} \hat{y}(v, w)$. Here $F_\varphi(w_0|v)$ is the set of candidate alternatives at variable w_0 after the local refinement F_φ , restricted to $F_\varphi(y_0) = v$. As refinement proceeds, some candidates w may be removed from $F_\varphi(w_0|v)$, allowing the rule-level upper bound to be progressively tightened (see [4] for details).

Intuitively, when $\text{UB}_\varphi(u, v)$ decreases by $\Delta_{\text{UB}_\varphi}$, the candidate-level upper bound $\hat{s}(u, v, w)$ is reduced by $\lambda \cdot \frac{1}{|H|} \cdot \Delta_{\text{UB}_\varphi}$. The following result shows that our upper bound is sound for early pruning alternatives that cannot enter the top- k set of anchor v (see [4]).

Theorem 6: Given a user u and top- k anchors in $K(u)$, the global upper bound $\hat{s}(u, w)$ is sound for pruning non-top- k alternatives. $\square$

Algorithm. We present algorithm TopK in Figure 7. It returns a top- k anchor set $K(u)$ and the top- k alternatives for each $v \in K(u)$.

Algorithm TopK first computes $K(u)$ using TopKAnchor (Line 2), constructs the corresponding OMR group and candidate map (Lines 3-5), and initializes candidate-level bounds. During coupled

refinement, a candidate w is discarded whenever $\hat{s}(u, w) < \tau$, where τ is the current k th-best score; otherwise its exact score and affected bounds are updated (Lines 9-21). The final heap forms $K_d(u)$.

Example 5: Consider a user u with $k = 2$, $\lambda = 0.5$, and three candidate anchors v_1, v_2 and v_3 . TopKAnchor returns $K(u) = \{v_1, v_2\}$. Initially, $\Psi = \emptyset$ and $\tau = 0$. After local refinement with φ_1 , candidates w_1, w_2 and w_3 are generated. Processing w_1 , it is not pruned; (v_1, w_1) yields $s(u, v_1, w_1) = 0.6$, so $\text{best}_{w_1} = 0.6$ and $\Psi = \{(w_1, 0.6)\}$. Next, w_2 has $\hat{s}(u, w_2) = 0.533$ and is retained since $|\Psi| < k$; (v_1, w_2) gives $s(u, v_1, w_2) = 0.55$, updating $\text{best}_{w_2} = 0.55$ and $\Psi = \{(w_1, 0.6), (w_2, 0.55)\}$ with $\tau = 0.55$. In contrast, w_3 is pruned since $\hat{s}(u, w_3) = 0.5 < \tau$. As refinement proceeds, bounds tighten until group refinement reaches a fixpoint. $\square$

Complexity. TopK takes $O(T_{\text{Anchor}} + |\Sigma_{\text{OMR}}| + |Q^H| + t|H|c_{\text{match}})$ time in total, since (a) TopKAnchor takes T_{Anchor} time, (b) it takes $O(|\Sigma_{\text{OMR}}| + |Q^H|)$ time to construct the rule group and initialize the candidate map; and (c) iterative group refinement costs $O(t|H|c_{\text{match}})$, where c_{match} is the cost of a single local refinement in TopK. Note that $c_{\text{match}} \ll |Q|_{\max}|G|$, as many unnecessary candidates are pruned by the global upper bounds.

Remark. Weak coupling limits alternative generation to top- k anchors, batch validation avoids repeated TIE evaluation, and score bounds avoid full alternative scoring. All pruning is safe, so TopK returns the same answers as exhaustive materialization and sorting.

6 EXPERIMENTAL STUDY

Using real-life graphs, we empirically evaluated the effectiveness and efficiency of OpenMind in CTR and top- k recommendations.

Experimental setting. We start with the experimental setting.

Experimental data. We used three user-item interaction graphs in real-life. (1) Anime [1], a bipartite user-anime review network that contains 47K users, 16K anime and 130K reviews (edges). Anime was enriched with features from both user profiles and anime meta-data, e.g., demographics and genres. (2) Steam [2], a large-scale bipartite user-game graph that contains 14.3M users, 2.6M games and 41M edges. We augmented the game vertices with extra metadata, e.g., pricing and tags. (3) Book [3], a large-scale bipartite user-book interaction network that contains 808K users, 1.56M books and 225M edges. Book has multiple user behaviors.

To evaluate the scalability, we implemented a graph generator to produce larger synthetic graphs of billion-scale. They were controlled by the number of vertices (up to 0.1B) and edges (up to 0.9B).

Following [28, 29, 99], we randomly split each user’s interaction history into 70%/10%/20% for training/validation/testing. Observed interactions were positive instances; while a negative instance paired the user with an item not previously interacted with.

Baselines. We compared OpenMind with six state-of-the-art models for diversified recommendations: (1) DPP [14], using *Determinantal Point Process* for relevant and diverse recommendations; (2) AdpMMR [24], computing user diversity propensity via feature entropy; (3) PDGAN [103], a personalized DPP with learned kernel matrix; (4) RAPID [68], incorporating personalized diversity with topic representation during training; (5) RecLogic [28], generating only anchors by TIEs; and (6) Seq, which sequentially finds

anchor-alternative pairs by first applying TIEs and then OMRs. For rule-based methods, we used DeepFM [46] as $\mathcal{M}_{\text{rec}}$.

We do not compare with influence maximization methods [9, 43, 53, 94] since they address network-wide polarization mitigation rather than personalized diverse recommendations. Other diversity-aware re-ranking models are excluded due to their poor performance compared with the selected models above [68].

Rules. We mined 436 (resp. 156), 567 (resp. 196), and 515 (resp. 206) OMRs (resp. TIEs) on Anime, Steam and Book, respectively.

Default parameters. By default, we set $\lambda = 0.6$ for score computations, and $k = 10$ for top- k recommendation. For the fuzzy area of $\mathcal{M}_{\text{rec}}$, δ_L and δ_H were automatically decided on validation datasets, yielding δ_H (resp. δ_L) as 0.61 (resp. 0.40), 0.68 (resp. 0.47) and 0.56 (resp. 0.31) on Anime, Steam and Book, respectively.

Evaluation metrics. We evaluated four commonly used metrics [25, 28, 40, 46, 96, 102, 112] for CTR recommendation. We evaluated each metric on n randomly selected testing users. We set $n = 1000$.

(1) *Area Under the ROC Curve* (AUC). It measures recommendation accuracy, defined as the probability that a method returns a higher score for a random positive sample than for a random negative one.

(2) *Precision* (Precis). It measures the ratio of true positives to all the decisions made by the recommendation method, defined as: $\text{Precis} = 1/n \cdot \sum_u \#TP_u / |\mathcal{R}_u|$, where $\mathcal{R}_u$ is the recommendation list for user u and $\#TP_u$ is the number of true positives at u . To be fair, we set $\mathcal{R}_u$ for OpenMind by merging two sets $I(u)$ and $I_d(u)$.

(3) *Intra-List Diversity* (ILD). It evaluates the pairwise item diversity within one recommendation list, defined as follows:

$$\text{ILD} = \frac{1}{n} \sum_u \frac{1}{|\mathcal{R}_u| \cdot (|\mathcal{R}_u| - 1)} \sum_{i \in \mathcal{R}_u} \sum_{j \in \mathcal{R}_u \setminus \{i\}} d_{ij},$$

where d_{ij} is the *Hamming distance* between two items (i, j) [58].

(4) *Filter Bubble Penetration* (FBP). It evaluates the proportion of recommended items outside users’ echo chambers, defined as:

$$\text{FBP} = \frac{1}{n} \sum_u \frac{|\text{Out}(\mathcal{R}_u)|}{|\mathcal{R}_u|},$$

where $\text{Out}(\mathcal{R}_u)$ is the set of items outside u ’s echo chamber, i.e., items whose average similarity score to the items u historically interacted with is below threshold β (see [4] for how to determine β).

For top- k recommendation, we adopt two widely used metrics to evaluate accuracy [28, 68]. (1) *Recall@ k* [28], defined as the fraction of ground-truth positive items that appear in the top- k recommendation list. (2) *NDCG@ k* [56], which evaluates ranking quality by assigning higher weights to relevant items appearing at higher positions in the top- k list. For diversity, we measure *ILD@ k* and *FBP@ k* by instantiating $\mathcal{R}_u$ as the top- k recommendation list in their respective definitions. We vary $k \in \{10, 20, 30, 40, 50\}$.

Implementation of $\mathcal{M}_{\text{dis}}$. We instantiate the pluggable predicate $\mathcal{M}_{\text{dis}}(y, w)$ with Random Walk with Restart (RWR) [66]. For each item pair (y, w) , we extract a local interaction subgraph, initialize RWR from authority vertices associated with each item, and use the separation between the resulting walk distributions as their dissimilarity score. Larger values indicate stronger viewpoint separation.

We choose RWR since filter bubbles often manifest as community separation in interaction graphs, which random-walk reachability

Methods	Anime				Steam				Book			
	AUC	Precis	ILD	FBP	AUC	Precis	ILD	FBP	AUC	Precis	ILD	FBP
AdpMMR	0.6876	0.3228	0.2441	0.1945	0.7589	0.3782	0.1274	0.1379	0.7829	0.3743	0.1387	0.2054
DPP	0.6621	0.321	0.2778	0.2634	0.7449	0.3712	0.1536	0.1591	0.7612	0.3688	0.1896	0.2369
PDGAN	0.6753	0.3202	0.2712	0.2516	0.7238	0.3468	0.1375	0.1408	0.7782	0.3569	0.1709	0.2237
RAPID	0.6882	0.3264	0.2818	0.2719	0.7519	0.3629	0.1539	0.1396	0.7894	0.3731	0.1678	0.2547
RecLogic	0.7292	0.3584	0.1918	0.1325	0.7802	0.4217	0.0764	0.0798	0.8163	0.4134	0.119	0.1681
Seq	0.6861	0.3282	0.3254	0.3869	0.7618	0.3727	0.1975	0.205	0.7882	0.3767	0.2412	0.3093
OpenMind	0.6861	0.3282	0.3254	0.3869	0.7618	0.3727	0.1975	0.205	0.7882	0.3767	0.2412	0.3093

Table 1: Effectiveness of CTR recommendations (the best results are in bold).

Methods	Anime				Steam				Book			
	Recall@10	NDCG@10	ILD@10	FBP@10	Recall@10	NDCG@10	ILD@10	FBP@10	Recall@10	NDCG@10	ILD@10	FBP@10
AdpMMR	0.5718	0.5101	0.2438	0.1973	0.3116	0.4948	0.1235	0.1348	0.4106	0.5301	0.1328	0.2075
DPP	0.55	0.5034	0.2741	0.2515	0.3079	0.4869	0.1493	0.1548	0.398	0.5258	0.1833	0.2296
PDGAN	0.5712	0.5078	0.2799	0.2508	0.2984	0.4862	0.1394	0.1442	0.4118	0.5432	0.1751	0.2262
RAPID	0.5897	0.5222	0.2836	0.2451	0.3048	0.4874	0.1562	0.1436	0.4185	0.5372	0.1826	0.2471
Seq	0.5834	0.5136	0.3233	0.3754	0.3193	0.4996	0.2139	0.2174	0.4199	0.5399	0.2562	0.2865
OpenMind	0.5834	0.5136	0.3233	0.3754	0.3193	0.4996	0.2139	0.2174	0.4199	0.5399	0.2562	0.2865

Table 2: Effectiveness of top- k recommendations where $k = 10$ (the best results are in bold).

captures naturally. RWR is not inherent to OpenMind: we also instantiate $\mathcal{M}_{dis}$ with cosine similarity and SimCLR, and vary the restart probability. As shown in Exp-7, OpenMind remains effective across $\mathcal{M}_{dis}$ instantiations and stable across RWR settings; Exp-6 further validates ILD and FBP vs. human judgments.

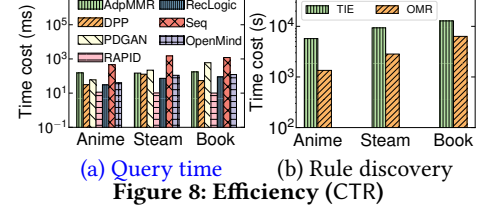
Environment. The experiments were conducted on a single machine powered by 256GB RAM, 32 processors with Intel(R) Xeon(R) Gold 5320 CPU @2.20GHz and one NVIDIA TESLA V100 GPU with 32 GB memory. OpenMind is implemented mainly in C++. For space limit, we report results on some datasets; the results on the others are consistent. We run each experiment three times and report the average.

Experimental results. We next report our findings.

Exp-1: Effectiveness of CTR recommendation. As shown in Table 1, the accuracy of OpenMind (AUC and Precision) is comparable to SOTA baselines. This is expected, as it optimizes a multi-objective tradeoff between relevance and viewpoint diversity, but AUC/Precision capture only relevance. By explicitly incorporating dissimilar yet acceptable alternatives, OpenMind operates on a different point of the Pareto frontier, to improve diversity with minimal loss in accuracy. Hence, comparable accuracy indicates that OpenMind preserves relevance while effectively expanding the solution space beyond single-objective recommenders.

For diversity, OpenMind consistently outperforms all baselines. Across the three datasets, its ILD exceeds AdpMMR, DPP, PDGAN, RAPID and RecLogic by 54.08%, 24.31%, 34.92%, 29.18% and 110.28% on average, respectively, up to 73.90%, 28.58%, 43.64%, 43.74% and 158.51%. Its corresponding FBP gains are 66.05%, 35.43%, 45.88%, 36.86% and 144.30%. These gains reflect different diversity objectives. Re-ranking methods are limited by pre-selected candidate pool and mainly diversify topics; AdpMMR, DPP and PDGAN further emphasize item features while largely ignoring interaction structure; RAPID is constrained by the base recommender’s biased candidates. In contrast, OpenMind explicitly generates viewpoint-diverse alternatives by combining graph interactions and feature signals with OMRs. RecLogic does not generate alternatives, while Seq achieves similar effectiveness but is much slower (Exp-2).

Varying λ . We varied the trade-off parameter $\lambda \in [0, 1]$ from acceptability- to diversity-focused. As λ increases, the accuracy (AUC, Precis) decrease while the diversity (ILD, FBP) increase (Figure 9(a)), reflecting the relevance-diversity tradeoff. Thus, λ selects a point that balances preference alignment and dissimilarity. On Steam, $\lambda = 0.6$ strikes the best balance, with strong diversity and



minimal accuracy loss. Similar results are observed for Anime (not shown) and Book (Figure 9(b)). This justifies the need for joint optimization rather than treating each objective in isolation.

Exp-2: Efficiency of CTR recommendation. We evaluated the average runtime per user of CTR in OpenMind. As shown in Figure 8(a), OpenMind is up to 13.7× faster than Seq across all datasets, as it performs coupled refinement over a shared candidate map, avoiding redundant enumeration and refinement of candidates in Seq. On Steam, OpenMind takes only 0.11 seconds to find all anchor-alternative pairs, whereas Seq takes 1.5 seconds.

OpenMind is faster than AdpMMR and PDGAN by up to 3.76× and 4.92× over three datasets, respectively, since the baselines use sequential greedy selection to optimize topic diversity, while OpenMind incrementally updates scores in the coupled refinement process. It is comparable to DPP as DPP computes diversity for only a few selected items, instead of all items. It adds negligible overhead to RecLogic despite computing alternatives. It is slower than RAPID, which benefits from efficient end-to-end neural inference.

Varying $|\Sigma_{OMR}|$ and $|\Sigma_{TIE}|$. We varied the size of Σ_{OMR} (resp. $|\Sigma_{TIE}|$) in Figure 9(c) (resp. Figure 9(d)). As expected, OpenMind takes longer with more OMRs/TIEs, yet it remains efficient, handling hundreds of OMRs/TIEs within about 0.12 seconds on Book.

Varying $|G|$. We varied the scaling factor of G from 20% to 100% on the synthetic graphs. All methods take longer for a larger graph. In Figure 9(e), CTR is 7.37× slower when $|G|$ is from 20% to 100%, indicating that its runtime grows steadily with graph size.

Exp-3: Effectiveness of TopK recommendation. Table 2 reports results for $k = 10$. On average over three datasets, OpenMind beats the baselines by 3.10%, 1.29%, 42.19% and 45.98% in Recall@10, NDCG@10, ILD@10 and FBP@10, respectively. On Book, e.g., its FBP is better than the best baseline RAPID by up to 15.94%, validating its effectiveness in breaking filter bubbles. Although RAPID has higher Recall@10 and NDCG@10 on Anime, its diversity is much worse: lower ILD@10 and FBP@10 by 14% and 53.16%, respectively.

Varying λ . We varied λ from 0 to 1 on Anime (Figure 9(f)). As λ increases, the diversity metrics ILD and FBP improve while Recall and NDCG slightly decrease, reflecting the diversity-acceptability tradeoff. Thus, λ controls the balance between diversity and accuracy; similar for Steam and Book (not shown).

Varying k . We varied k from 10 to 50. As shown in Figure 9(g), ILD

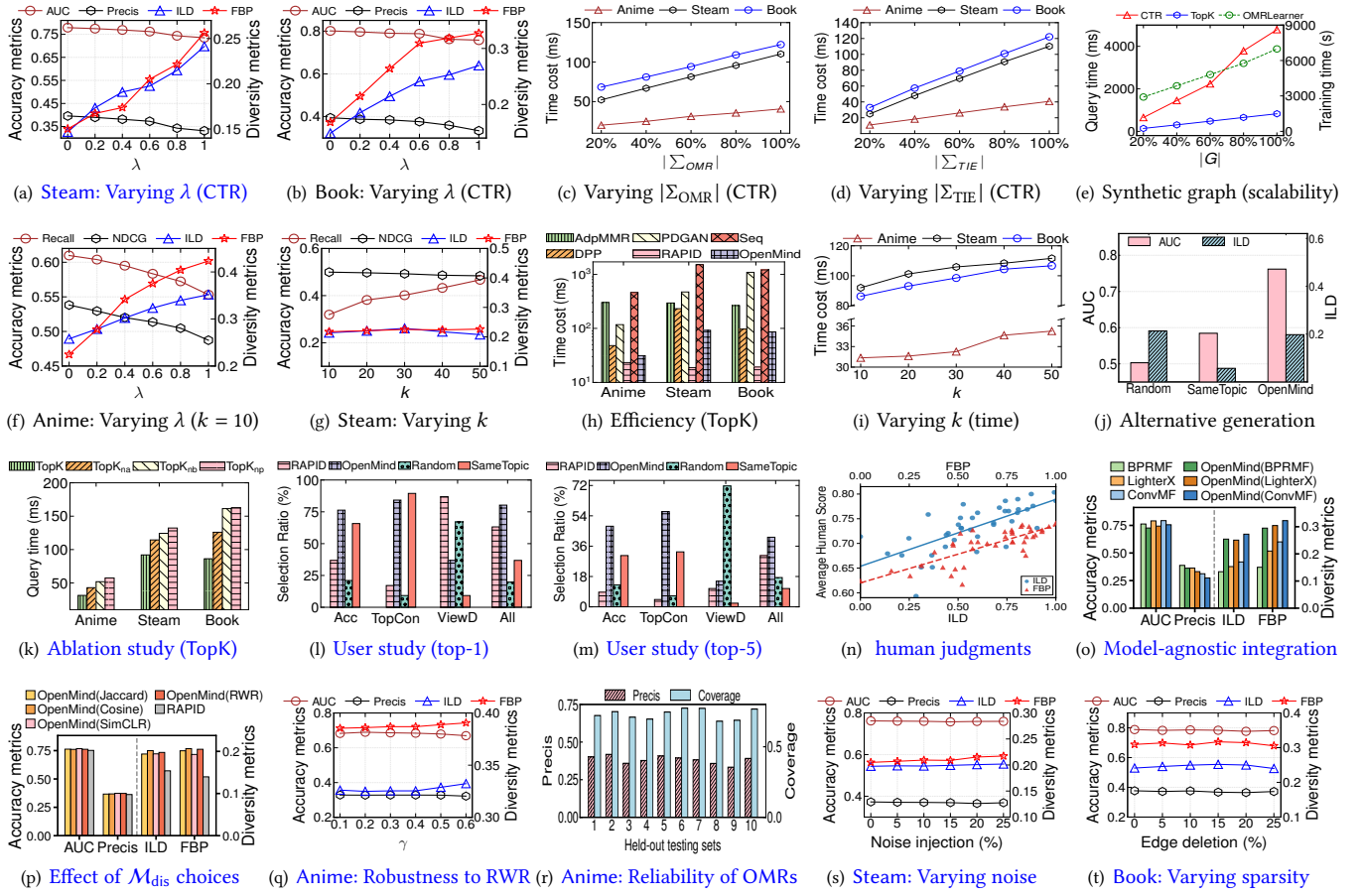


Figure 9: Performance evaluation

and FBP on Steam remain stable, indicating that the ranking score consistently prioritizes more promising alternatives. Meanwhile, Recall@ k increases with k as more relevant items are included, while NDCG@ k remains stable, showing that top-ranked quality is preserved. Similar trends hold on Anime and Book (not shown).

Exp-4: Efficiency of top- k recommendation. We report the average per-user runtime for top- k recommendation in Figure 9(h). On Book, OpenMind takes 86 ms on average. (1) OpenMind is much faster than DPP, AdpMMR, PDGAN and Seq, by up to 2.5 $\times$, 9.6 $\times$, 12.6 $\times$ and 16.4 $\times$, respectively. This is because the baselines rely on sequential greedy selection, while TopK in OpenMind employs multiple pruning strategies to avoid unnecessary computations. (2) OpenMind is slower than RAPID, which uses an end-to-end neural model to score all items and directly return the top- k results. However, RAPID incurs higher offline training cost.

Varying k . As shown in Figure 9(i) by varying k from 10 to 50 on Book, TopK incurs longer time as k increases due to the need to evaluate more candidates. On the largest graph Book, the runtime increases by only 20.4 ms as k increases from 10 to 50.

Varying $|G|$. We varied the scaling factor of the synthetic graph G from 20% to 100%. All methods take longer on larger graphs. In Figure 9(e), TopK is 5.99 $\times$ slower when $|G|$ is from 20% to 100%.

Varying $|\Sigma_{OMR}|$ and $|\Sigma_{TIE}|$. We varied the sizes of Σ_{OMR} and $|\Sigma_{TIE}|$. TopK takes longer with more OMRs or TIEs (not shown).

Exp-5: Ablation study. We conducted ablation studies to evaluate

the contribution of each component in OpenMind.

Effectiveness of alternative generation. We compared OpenMind with two variants on generating alternatives for CTR recommendation: Random that randomly samples alternatives from all items, and SameTopic that randomly selects alternatives from items of the same topic as the anchor. In Figure 9(j), OpenMind achieves 30.33% and 247.71% higher AUC and ILD than SameTopic on Steam, respectively, verifying the effectiveness of viewpoint-aware alternative generation. Compared with Random, OpenMind improves the AUC by 51.45%, as Random selects items that are misaligned with the user’s interests, confirming the need for user acceptability.

CTR. We compared CTR in OpenMind with its variant CTR_{nu} that naively computes the score for each pair without using updateScore. CTR runs faster than CTR_{nu} by up to 10 $\times$ on Book, verifying the effectiveness of incrementally updating scores (not shown).

TopK. We compared TopK with three variants: (1) TopK_{na}, which enumerates all anchor-alternative pairs via CTR, ranks them by score, and returns the top- k results; (2) TopK_{nb}, which follows the same pipeline as TopK but replaces batch validation with per-item TIE evaluation; and (3) TopK_{np}, which adopts the same anchor selection as TopK but disables upper-bound pruning for alternatives.

In Figure 9(k), OpenMind is 1.56 $\times$ faster than all baselines on average when $k = 10$. It beats TopK_{na}, TopK_{nb} and TopK_{np} by up to 1.46 $\times$, 1.87 $\times$ and 1.89 $\times$, respectively, validating the effectiveness of TopK’s design choices, including weakly coupled stages, batch validation for anchor selection, and early pruning for alternatives.

Exp-6: User studies. We conducted three user studies on Anime and Steam to assess whether OpenMind mitigates filter bubbles in practice. We compared OpenMind with RAPID, Random, and SameTopic using four metrics: (1) *Acceptability* (Acc), whether users would accept the recommendations; (2) *Topic Consistency* (TopCon), whether they remain on the same topic; (3) *View Divergence* (ViewD), whether they present different viewpoints; and (4) *Overall* (All), which jointly considers the above criteria (see [4] for more).

Evaluation settings. We recruited 90 participants, aged 18–44, from several universities and research institutes, including graduate students, research staff, and software engineers in computer science. All were familiar with online recommendation scenarios and filter bubbles. For quality control, participants selected both the best and the best two recommendations for each metric; a response was discarded if the latter did not include the former.

Top-1 recommendation. The top-1 recommendation of each evaluated method on Anime is provided, each showing the user profile, favorite anime, most recently watched anime, and details of anime. In total, we collected 76 (out of 84) valid responses. In Figure 9(l), 61 of the responses indicate that OpenMind achieves the best overall performance. Although RAPID and Random have higher ViewD than OpenMind as both of them provide across-topic items, they sacrifice the overall performance. Compared with RAPID (resp. Random), OpenMind achieves 39.48% (resp. 55.27%), 67.1% (resp. 75%) and 17.1% (resp. 60.52%) higher Acc, TopCon and All, respectively, verifying the effectiveness of balancing both acceptability and diversity in alternative generation. Compared with SameTopic, OpenMind improves ViewD by 27.63%, showing that the topic preservation alone is insufficient for exposing users to opposing views.

Top-5 recommendations. We conducted another user study by providing top-5 recommendations on Steam. In total, we collected 46 (out of 60) valid responses. In Figure 9(m), OpenMind improves ViewD by 13.05% (resp. 4.35%) over SameTopic (resp. RAPID). Although Random attains the highest ViewD, its Acc, TopCon and All fall below OpenMind by 34.79%, 50.00% and 23.91%, respectively, indicating Random includes items users are unlikely to accept.

Validation via human judgment. We also conducted a metric-human correlation study to assess whether ILD and FBP reflect perceived viewpoint diversity. We recruited 58 annotators from a research institute, all experienced with online content platforms. Each annotator rated 40 recommendation lists produced by OpenMind on a 5-point Likert scale for (a) perceived viewpoint diversity and (b) exposure to perspectives beyond prior beliefs. We then computed Spearman’s rank correlation [87] between the automatic metrics and human judgments. As shown in Figure 9(n), ILD and FBP achieve correlations of 0.76 and 0.74, respectively, with the statistical significance $p < 0.001$ of each correlation (*i.e.*, the observed correlations are extremely unlikely to occur by random chance), confirming that they meaningfully reflect user perceptions.

Exp-7: Effect of ML predicates. We evaluated the effect of ML predicates used in OMRs for the diverse recommendation.

Model-agnostic integration. For the model generality of OpenMind, we considered three additional recommenders beyond RecLogic: (a) BPRMF [81], a collaborative-filtering model; (b) LighterX [114],

a recent GNN-based model; and (c) ConvMF [59], a hybrid model.

As shown in Figure 9(o), on Book, OpenMind improves ILD and FBP by 71.05% and 57.48% on average across all base recommenders while maintaining comparable accuracy, supporting its model-agnostic integration. Its query-processing time remains within 47–219 ms, indicating that the same CTR algorithm apply without recommender-specific modifications (not shown).

Effect of M_{dis} choices. To assess sensitivity to the choice of M_{dis} , we additionally instantiated it with (a) Jaccard distance [13], (b) cosine similarity [62] and (c) SimCLR [15]. As shown in Figure 9(p), on Steam, OpenMind consistently beats the strongest baseline, RAPID, across all M_{dis} instantiations, improving average ILD (resp. FBP) by 27.86% (resp. 44.27%). Similarly on Anime and Book (not shown).

Robustness to RWR parameters. We varied the restart probability γ from 0.1 to 0.6 on Anime. As shown in Figure 9(q), OpenMind remains stable as γ increases, with FBP varying by less than 0.49%. This robustness improves because M_{dis} thresholds absorb moderate RWR-score fluctuations, and moreover, the other conditions in each OMR jointly reduce sensitivity to any single M_{dis} parameter. Thus, OpenMind is not brittle to RWR settings.

Exp-8: Quality of OMRs. We also evaluated the reliability, robustness, and diversity of the discovered OMRs.

Coverage and reliability. On Anime, we applied OMRs mined from training data to ten randomly sampled subsets of the held-out testing set. We first measured coverage, defined as the fraction of test user-item pairs covered by at least one OMR. In Figure 9(r), OMRs cover 73.06% of test pairs on average; recommendation precision changes by less than 7.97%, indicating good generalization. Moreover, 59 annotators judged 20 randomly sampled anchor-alternative pairs. The precisions for topical relatedness, viewpoint difference, and acceptability are 0.7407, 0.8186 and 0.7915, respectively (0.7836 on average), confirming semantic validity.

Robustness and stability. We tested noisy and sparse interactions by injecting 5%–25% spurious edges on Steam and deleting 5%–25% of the original edges on Book, respectively. At each setting, we mined the OMRs and measured downstream quality. In Figure 9(s), under 25% noise, AUC drops by less than 0.29% on Steam. In Figure 9(t), even at 25% edge deletion, AUC decrease by only 0.86% on Book. Across 10 random training splits, the mined rule sets achieve 0.7942 rule-level Jaccard similarity and 0.8289 consistency in covered test pairs (see [4] for figures). Thus, OMR discovery is robust to noisy and sparse interactions and stable across training samples.

Diversity. We evaluated whether the discovered OMRs Σ_{OMR} capture complementary patterns via three dimensions: (1) *structural diversity*, measured by average pairwise pattern dissimilarity; (2) *predicate diversity*, measured by predicate-set dissimilarity; and (3) *answer diversity*, measured by the average Jaccard distance between covered test pairs. To control for rule-set size, we compared two equally sized subsets: Σ_{OMR}^{sup} , containing the highest-support OMRs, and Σ_{OMR}^{rand} , sampled uniformly from Σ_{OMR} . On average, Σ_{OMR}^{sup} achieves 6.79%, 26.36% and 58.72% lower structural, predicate, and answer diversity, respectively, than Σ_{OMR}^{rand} on all datasets (see [4] for figure), indicating that high-support rules capture dominant patterns. Moreover, Σ_{OMR} covers 6.72% more test pairs than Σ_{OMR}^{sup} .

and its coverage gets larger as additional OMRs are included (not shown), showing that lower-support rules contribute complementary rather than redundant patterns, so Σ_{OMR} as a whole is diverse.

Exp-9: Efficiency and scalability of rule discovery. Figure 8(b) shows that OpenMind consistently mines OMRs faster than TIEs. On Anime, discovery takes 0.37 hours for OMRs, versus 1.59 hours for TIEs, because OpenMind extends pre-mined TIEs rather than mining from scratch. In Figure 9(e), on synthetic graph, discovery is only 2.40 $\times$ slower when graph size is from 20% to 100%, confirming practical scalability. Discovery is performed once offline.

Summary. We find the following. (1) OpenMind provides high-quality CTR recommendation with diverse viewpoints; its ILD and FBP outperform prior methods by 35.62% and 46.06% on average, up to 110.28% and 144.30%, respectively, while retaining comparable accuracy. (2) For top- k recommendation, on average, Recall@ k , NDCG@ k , ILD@ k and FBP@ k improve by 3.10%, 1.29%, 42.19% and 45.98%, respectively, up to 7.0%, 2.76%, 92.92% and 90.27%. (3) OpenMind is efficient, taking 0.12s per user on average for a CTR query on graphs with 225M edges, 13.7 $\times$ faster than Seq. For top- k , it takes 86ms on average, up to 16.4 $\times$ faster. (4) By user studies, OpenMind is judged more acceptable and viewpoint-diverse than baselines. ILD and FBP correlate with human judgments by 0.76 and 0.74. (5) It is model-agnostic: OpenMind improves ILD and FBP by 71.05% and 57.48% on average across all base recommenders while preserving accuracy and query time. (6) The discovered OMRs are reliable and stable under noise/sparsity (accuracy drops by <0.86%).

7 RELATED WORK

We categorize the related work as follows.

Filter bubbles. Prior work on the topic falls into two categories.

Re-ranking. Given a user u and an initial list $\mathcal{R}_u$ from $\mathcal{M}_{\text{rec}}$, these methods reorder $\mathcal{R}_u$ to improve diversity, typically at the topic level. Representative approaches include MMR [12], DPP [14], and neural variants such as DESA [78] and SSD [54], all of which apply a uniform diversity strategy to all users; while personalized extensions (e.g., AdpMMR [24], PDGAN [103], RAPID [68]) incorporate diversification with user preferences. However, these methods operate on pre-selected candidates and diversify topics, limiting their ability to introduce contrasting viewpoints within the same context. Other methods (e.g., DLCM [6], SetRank [75], SRGA [77]) ignore diversity.

Balanced information exposure. Another line of work addresses polarization via influence propagation and network interventions [9, 43, 53, 72, 94], including cross-partisan edge construction [42] and user interface interventions [63, 64].

Recommendation systems. A host of recommendation models have been developed, including: (1) *CF-based models* that leverage user-item interactions [51, 60, 67, 70, 100], often enhanced by deep learning architectures such as MLP [17, 26], autoencoders [83, 88], CNN [50, 93] and GNN [49, 100, 114]; (2) *content-based models* that exploit auxiliary features (e.g., images and text) to profile users [69, 101, 106]; (3) *hybrid models* that combine multiple strategies [5, 16, 18, 90, 95, 113]; and (4) *rule-based frameworks* that augment recommendation with logical reasoning, allowing arbitrary models to be plugged in, e.g., RecLogic [28]. However, these models primarily optimize accuracy and do not explicitly account for diversity.

OpenMind differs from prior approaches along three dimensions. (1) *Viewpoint-level diversity.* Unlike diversification methods [12, 14, 22–24, 68, 103] that primarily focus on *topic-* or *category-level* diversity, to broaden user exposure across topics, OpenMind targets *viewpoint diversity*, i.e., exposing users to alternative or opposing views on the same topic. (2) *Individual-level diversification.* Prior work on information chambers [9, 42, 43, 63, 64, 72, 94] either studies user reactions to opposing views [63, 64] or optimizes placement and targeting to maximize influence in information propagation [9, 42, 43, 72, 94]. However, they operate at the population level. Instead, OpenMind generates a personalized complementary set $I_d(x)$ for each user x at query time. (3) *Rule-based generation.* OpenMind uses OMRs to jointly encode topical relatedness, viewpoint difference, relevance, and engagement, evaluating anchors and alternatives through coupled query-time inference rather than post-hoc retrieve-then-diversify; it further extends RecLogic [28] to top- k recommendation for both anchors and alternatives.

Rules for graphs. Prior work develops graph rules for various tasks, including graph dependencies and inconsistency handling [30, 36, 39], association and event rules for scalable discovery and prediction [31, 33, 34], error detection [38], and explanation [29]. Unlike these methods, OpenMind applies logical graph rules at query time to generate viewpoint-diverse recommendations.

Subgraph/pattern matching. Subgraph matching for a single pattern has been accelerated via relaxed semantics [35], matching-order optimization and indexing [11, 37, 48], compact indexes [10], and distributed enumeration over large graphs [61, 79, 89, 91, 111]. In contrast, OpenMind must jointly evaluate two interdependent rule classes, TIEs for anchors and OMRs for alternatives, requiring shared inference rather than independent single-pattern matching.

Multi-query optimization. It reduces time cost by sharing computation across queries via sub-expression reuse [84], heuristic sharing or caching [80, 82, 98]. Unlike these, OpenMind jointly evaluates interdependent rule sets over a shared candidate map.

Top- k queries. Threshold-based aggregation [21], progressive merging for ad-hoc ranking [105], and concurrent preference queries [109] enable efficient top- k retrieval; see [55] for a survey. While OpenMind follows the top- k paradigm [27], its rule-based setting requires new mechanisms, including batched validation and score bounds, to compute both top- k anchors and top- k alternatives.

8 CONCLUSION

We have studied how to augment recommender systems with alternative viewpoints to mitigate filter bubbles while preserving relevance. We have proposed OpenMind, a lightweight rule-based framework that integrates logical reasoning with ML signals to generate complementary recommendations. OpenMind supports both CTR and top- k settings, coupling anchor selection and alternative generation in a single inference pass. Our analytical and empirical results have shown that viewpoint-aware recommendation can be achieved efficiently without modifying underlying models, offering a practical path to mitigating filter bubbles in real-world systems.

Future work includes cost-based scheduling for OMR discovery, rule weighting to balance diversity and efficiency at scale, deployment and online A/B testing of OpenMind in real recommenders.

REFERENCES

- [1] 2024. Anime Dataset with Reviews - MyAnimeList. <https://www.kaggle.com/datasets/marlesson/myanimelist-dataset-animes-profiles-reviews>.
- [2] 2024. Game Recommendations on Steam. <https://www.kaggle.com/datasets/antonkozryiev/game-recommendations-on-steam/>.
- [3] 2024. Goodreads Book Reviews. <https://www.kaggle.com/datasets/pypiahmad/goodreads-book-reviews1/>.
- [4] 2026. OpenMind: Query-Time Diversification via Logical Rules (full). https://github.com/hdqlkncso24jvf/OpenMind/blob/main/OpenMind_full_version.pdf.
- [5] Sajad Ahmadian, Nima Joorabloo, Mahdi Jalili, Majid Meghdadi, Mohsen Afsharchi, and Yongli Ren. 2018. A temporal clustering approach for social recommender systems. In *ASONAM*. IEEE, 1139–1144.
- [6] Qingyao Ai, Keping Bi, Jiafeng Guo, and W Bruce Croft. 2018. Learning a deep listwise context model for ranking refinement. In *SIGIR*. 135–144.
- [7] Heba Al-Mamouri and Wadhah R Baiee. 2023. Predicting Content Popularity on Social Media: An Analytical Approach Using Regression Modeling. In *ICCCN*. Springer, 649–659.
- [8] Eytan Bakshy, Solomon Messing, and Lada A Adamic. 2015. Exposure to ideologically diverse news and opinion on Facebook. *Science* 348, 6239 (2015), 1130–1132.
- [9] Prithu Banerjee, Wei Chen, and Laks V. S. Lakshmanan. 2023. Mitigating Filter Bubbles Under a Competitive Diffusion Model. *SIGMOD* 1, 2 (2023), 1–26.
- [10] Bibek Bhattarai, Hang Liu, and H. Howie Huang. 2019. CECI: Compact Embedding Cluster Index for Scalable Subgraph Matching. In *SIGMOD*.
- [11] Fei Bi, Lijun Chang, Xuemin Lin, Lu Qin, and Wenjie Zhang. 2016. Efficient subgraph matching by postponing cartesian products. In *SIGMOD*. 1199–1214.
- [12] Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *SIGIR*. 335–336.
- [13] Surajit Chaudhuri, Venkatesh Ganti, and Raghav Kaushik. 2006. A primitive operator for similarity joins in data cleaning. In *ICDE*. IEEE, 5–5.
- [14] Laming Chen, Guoxin Zhang, and Eric Zhou. 2018. Fast greedy map inference for determinantal point process to improve recommendation diversity. *NeurIPS* 31 (2018).
- [15] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. SimCLR: A simple framework for contrastive learning of visual representations. In *ICLR*, Vol. 2.
- [16] Xu Chen, Yongfeng Zhang, and Zheng Qin. 2019. Dynamic explainable recommendation based on neural attentive models. In *AAAI*, Vol. 33. 53–60.
- [17] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishu Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Isipir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & deep learning for recommender systems. In *DLRS*. 7–10.
- [18] Wei-Ta Chu and Ya-Lun Tsai. 2017. A hybrid recommendation system considering visual information for predicting favorite restaurants. *WWW* 20, 6 (2017), 1313–1331.
- [19] Aaron Clauset, Cosma Rohilla Shalizi, and Mark EJ Newman. 2009. Power-law distributions in empirical data. *SIAM Rev.* 51, 4 (2009), 661–703.
- [20] Elanor Colleoni, Alessandro Rozza, and Adam Arvidsson. 2014. Echo chamber or public sphere? Predicting political orientation and measuring political homophily in Twitter using big data. *Journal of communication* 64, 2 (2014), 317–332.
- [21] Gautam Das, Dimitrios Gunopulos, Nick Koudas, and Dimitris Tsirgiannis. 2006. Answering top-k queries using views. In *PVLDB*. 451–462.
- [22] Ting Deng and Wenfei Fan. 2014. On the Complexity of Query Result Diversification. *TODS* 39, 2 (2014), 15:1–15:46.
- [23] Ting Deng, Wenfei Fan, and Floris Geerts. 2013. On the Complexity of Package Recommendation Problems. *SIAMJC* 42, 5 (2013), 1940–1986.
- [24] Tommaso Di Noia, Vito Claudio Ostuni, Jessica Rosati, Paolo Tomeo, and Eugenio Di Sciascio. 2014. An analysis of users’ propensity toward diversity in recommendations. In *RecSys*. 285–288.
- [25] Patrik Dokoupil, Ludovico Boratto, and Ladislav Peska. 2025. How Do Users Perceive Recommender Systems’ Objectives?. In *RecSys*. 165–176.
- [26] Gintare Karolina Dziugaite and Daniel M Roy. 2015. Neural network matrix factorization. *arXiv preprint arXiv:1511.06443* (2015).
- [27] Ronald Fagin, Amnon Lotem, and Moni Naor. 2001. Optimal aggregation algorithms for middleware. In *SIGMOD*. 102–113.
- [28] Lihang Fan, Wenfei Fan, Ping Lu, Chao Tian, Muyang Liu, and Qiang Yin. 2023. Enriching Recommendation Models with Logic Conditions. *SIGMOD* 1, 3 (2023), 210.
- [29] Wenfei Fan, Lihang Fan, Dandan Lin, and Min Xie. 2024. Explaining GNN-based recommendations in logic. *PVLDB* 18, 3 (2024), 715–728.
- [30] Wenfei Fan, Wenzhi Fu, Ruochun Jin, Muyang Liu, Ping Lu, and Chao Tian. 2023. Making It Tractable to Catch Duplicates and Conflicts in Graphs. *SIGMOD* 1, 1 (2023), 86:1–86:28.
- [31] Wenfei Fan, Wenzhi Fu, Ruochun Jin, Ping Lu, and Chao Tian. 2022. Discovering Association Rules from Big Graphs. *PVLDB* 15, 7 (2022), 1479–1492.
- [32] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional functional dependencies for capturing data inconsistencies. *TODS* 33, 2 (2008), 6.
- [33] Wenfei Fan, Ruochun Jin, Muyang Liu, Ping Lu, Chao Tian, and Jingren Zhou. 2020. Capturing Associations in Graphs. *PVLDB* 13, 11 (2020), 1863–1876.
- [34] Wenfei Fan, Ruochun Jin, Ping Lu, Chao Tian, and Ruiqi Xu. 2022. Towards Event Prediction in Temporal Graphs. *PVLDB* 15, 9 (2022), 1861–1874.
- [35] Wenfei Fan, Jianzhong Li, Shuai Ma, Nan Tang, Yinghui Wu, and Yunpeng Wu. 2010. Graph pattern matching: From intractable to polynomial time. *PVLDB* 3, 1 (2010), 264–275.
- [36] Wenfei Fan, Xueli Liu, Ping Lu, and Chao Tian. 2018. Catching Numeric Inconsistencies in Graphs. In *SIGMOD*. 381–393.
- [37] Wenfei Fan, Yixuan Luo, Ping Lu, and Xiaoke Zhu. 2026. Enumerating Graph Pattern Matches with ML Oracles. *SIGMOD* 4, 1 (2026), 1–28.
- [38] Wenfei Fan, Kehan Pang, Ping Lu, and Chao Tian. 2024. Making it tractable to detect and correct errors in graphs. *TODS* 49, 4 (2024), 1–75.
- [39] Wenfei Fan, Yinghui Wu, and Jingbo Xu. 2016. Functional dependencies for graphs. In *SIGMOD*. ACM, 1843–1857.
- [40] Tom Fawcett. 2006. An introduction to ROC analysis. *Pattern recognition letters* 27, 8 (2006), 861–874.
- [41] Michael Garey and David Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company.
- [42] Kiran Garimella, Gianmarco De Francisci Morales, Aristides Gionis, and Michael Mathioudakis. 2017. Reducing controversy by connecting opposing views. In *WSDM*. 81–90.
- [43] Kiran Garimella, Aristides Gionis, Nikos Parotsidis, and Nikolaj Tatti. 2017. Balancing information exposure in social networks. *NeurIPS* 30 (2017).
- [44] Yingqiang Ge, Shuya Zhao, Honglu Zhou, Changhua Pei, Fei Sun, Wenwu Ou, and Yongfeng Zhang. 2020. Understanding echo chambers in e-commerce recommender systems. In *SIGIR*. 2261–2270.
- [45] Quentin Grossetti, Cédric Du Mouza, and Nicolas Travers. 2019. Community-based recommendations on Twitter: Avoiding the filter bubble. In *ICWISE*. Springer, 212–227.
- [46] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine based Neural Network for CTR Prediction. In *IJCAI*. 1725–1731.
- [47] Myoungji Han, Hyunjoon Kim, Geonmo Gu, Kunsu Park, and Wook-Shin Han. 2019. Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together. In *SIGMOD*. 1429–1446.
- [48] Wook-Shin Han, Jinsoo Lee, and Jeong-Hoon Lee. 2013. Turboiso: towards ultrafast and robust subgraph isomorphism search in large graph databases. In *SIGMOD*. 337–348.
- [49] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and powering graph convolution network for recommendation. In *SIGIR*. ACM, 639–648.
- [50] Xiangnan He, Xiaoyu Du, Xiang Wang, Feng Tian, Jinhui Tang, and Tat-Seng Chua. 2018. Outer Product-based Neural Collaborative Filtering. In *IJCAI*. 2227–2233.
- [51] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *WWW*. 173–182.
- [52] Cheng-Kang Hsieh, Longqi Yang, Yin Cui, Tsung-Yi Lin, Serge Belongie, and Deborah Estrin. 2017. Collaborative metric learning. In *WWW*. 193–201.
- [53] Shixun Huang, Wenqing Lin, Zhifeng Bao, and Jiachen Sun. 2022. Influence Maximization in Real-World Closed Social Networks. *PVLDB* 16, 2 (2022), 180–192.
- [54] Yanhua Huang, Weikun Wang, Lei Zhang, and Ruiwen Xu. 2021. Sliding spectrum decomposition for diversified recommendation. In *SIGKDD*. 3041–3049.
- [55] Ihab F Ilyas, George Beskales, and Mohamed A Soliman. 2008. A survey of top-k query processing techniques in relational database systems. *Comput. Surveys* 40, 4 (2008), 1–58.
- [56] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *TOIS* 20, 4 (2002), 422–446.
- [57] Minhao Jiang, Ada Wai-Chee Fu, and Raymond Chi-Wing Wong. 2017. READS: A random walk approach for efficient and accurate dynamic SimRank. *PVLDB* 10, 9 (2017), 937–948.
- [58] John Paul Kelly and Derek Bridge. 2006. Enhancing the diversity of conversational collaborative recommendations: a comparison. *Artificial Intelligence Review* 25, 1 (2006), 79–95.
- [59] Donghyun Kim, Chanyoung Park, Jinoh Oh, Sungyoung Lee, and Hwanjo Yu. 2016. Convolutional matrix factorization for document context-aware recommendation. In *ACM conference on recommender systems*. 233–240.
- [60] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [61] Longbin Lai, Lu Qin, Xuemin Lin, and Lijun Chang. 2015. Scalable subgraph enumeration in mapreduce. *PVLDB* 8, 10 (2015), 974–985.
- [62] Ken Lang. 1995. NewsWeeder: Learning to filter netnews. In *MLP*. Elsevier,

331–339.

- [63] Qingzi Vera Liao and Wai-Tat Fu. 2014. Can you hear me now?: Mitigating the echo chamber effect by source position indicators. In *CSCW*. ACM, 184–196.
- [64] Qingzi Vera Liao and Wai-Tat Fu. 2014. Expert voices in echo chambers: Effects of source expertise indicators on exposure to diverse opinions. In *CHI*. ACM, 2745–2754.
- [65] Dandan Lin, Shijie Sun, Jingtao Ding, Xuehan Ke, Hao Gu, Xing Huang, Chong-gang Song, Xuri Zhang, Lingling Yi, Jie Wen, and Chuan Chen. 2022. PlatoGL: Effective and scalable deep graph learning system for graph-enhanced real-time recommendation. In *CIKM*. 3302–3311.
- [66] Dandan Lin, Raymond Chi-Wing Wong, Min Xie, and Victor Junqiu Wei. 2020. Index-free approach with theoretical guarantee for efficient random walk with restart query. In *ICDE*. IEEE, 913–924.
- [67] Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet Computing* 7, 1 (2003), 76–80.
- [68] Weiwen Liu, Yunjia Xi, Jiarui Qin, Xinyi Dai, Ruiming Tang, Shuai Li, Weinan Zhang, and Rui Zhang. 2023. Personalized Diversification for Neural Re-ranking in Recommendation. In *ICDE*. IEEE, 802–815.
- [69] Beth Logan and Ariel Salomon. 2001. A content-based music similarity function. *Cambridge Research Labs-Tech Report* (2001).
- [70] Xin Luo, Mengchu Zhou, Yunni Xia, and Qingsheng Zhu. 2014. An efficient non-negative matrix-factorization-based approach to collaborative filtering for recommender systems. *TII* 10, 2 (2014), 1273–1284.
- [71] Stephann Makri, Ann Blandford, Mel Woods, Sarah Sharples, and Deborah Maxwell. 2014. "Making my own luck": Serendipity strategies and how to support them in digital information environments. *J. Assoc. Inf. Sci. Technol.* 65, 11 (2014), 2179–2194.
- [72] Antonis Matakos, Çigdem Aslay, Esther Galbrun, and Aristides Gionis. 2022. Maximizing the Diversity of Exposure in a Social Network. *TKDE* 34, 9 (2022), 4357–4370.
- [73] Dana McKay, Stephann Makri, Shanton Chang, and George Buchanan. 2020. On Birthing Dancing Stars: The Need for Bounded Chaos in Information Interaction. In *SIGIR-CHIIR*. ACM, 292–302.
- [74] Lien Michiels, Jens Leysen, Annelien Smets, and Bart Goethals. 2022. What are filter bubbles really? A review of the conceptual and empirical work. In *UMAP*. 274–279.
- [75] Liang Pang, Jun Xu, Qingyao Ai, Yanyan Lan, Xueqi Cheng, and Jirong Wen. 2020. SetRank: Learning a permutation-invariant ranking model for information retrieval. In *SIGIR*. 499–508.
- [76] Eli Pariser. 2011. *The filter bubble: What the Internet is hiding from you*. Penguin UK.
- [77] Hao Qian, Qintong Wu, Kai Zhang, Zhiqiang Zhang, Lihong Gu, Xiaodong Zeng, Jun Zhou, and Jinjie Gu. 2022. Scope-aware re-ranking with gated attention in feed. In *WSDM*. 804–812.
- [78] Xubo Qin, Zhicheng Dou, and Ji-Rong Wen. 2020. Diversifying search results using self-attention network. In *CIKM*. 1265–1274.
- [79] Xuguang Ren and Junhu Wang. 2015. Exploiting vertex relationships in speeding up subgraph isomorphism over large graphs. In *PVLDB*, Vol. 8. 617–628.
- [80] Xuguang Ren and Junhu Wang. 2016. Multi-query optimization for subgraph isomorphism search. *PVLDB* 10, 3 (2016), 121–132.
- [81] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2012. BPR: Bayesian personalized ranking from implicit feedback. *UAI* (2012).
- [82] Prasan Roy, Srinivasan Seshadri, S Sudarshan, and Siddhesh Bhobe. 2000. Efficient and extensible algorithms for multi query optimization. In *SIGMOD*. 249–260.
- [83] Suvash Sedhain, Aditya Krishna Menon, Scott Sanner, and Lexing Xie. 2015. AutoRec: Autoencoders meet collaborative filtering. In *WWW*. 111–112.
- [84] Timos K Sellis. 1988. Multiple-query optimization. *TODS* 13, 1 (1988), 23–52.
- [85] Yingdan Shang, Bin Zhou, Xiang Zeng, Ye Wang, Han Yu, and Zhong Zhang. 2022. Predicting the popularity of online content by modeling the social influence and homophily features. *Frontiers in Physics* 10 (2022), 915756.
- [86] Xiaoyu Shi, Quanliang Liu, Hong Xie, Di Wu, Bo Peng, MingSheng Shang, and Defu Lian. 2023. Relieving popularity bias in interactive recommendation: A diversity-novelty-aware reinforcement learning approach. *TOIS* 42, 2 (2023), 1–30.
- [87] Charles Spearman. 1961. The proof and measurement of association between two things. (1961).
- [88] Florian Strub, Romaric Gaudel, and Jérémie Mary. 2016. Hybrid recommender system based on autoencoders. In *DLRS*. 11–16.
- [89] Shixuan Sun and Qiong Luo. 2020. In-memory subgraph matching: An in-depth study. In *SIGMOD*. 1083–1098.
- [90] Zhu Sun, Qing Guo, Jie Yang, Hui Fang, Guibing Guo, Jie Zhang, and Robin Burke. 2019. Research commentary on recommendations with side information: A survey and research directions. *ECRA* 37 (2019).
- [91] Zhao Sun, Hongzhi Wang, Haixun Wang, Bin Shao, and Jianzhong Li. 2012. Efficient subgraph matching on billion node graphs. *PVLDB* 5, 9 (2012), 788–799.
- [92] Cass R. Sunstein. 2017. *Republic: Divided Democracy in the Age of Social Media*. Princeton University Press.
- [93] Jiayi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *WSDM*. 565–573.
- [94] Sijing Tu, Çigdem Aslay, and Aristides Gionis. 2020. Co-exposure Maximization in Online Social Networks. In *NeurIPS*.
- [95] Farman Ullah, Ghulam Sarwar, Sung Chang Lee, Yun Kyung Park, Kyeong Deok Moon, and Jin Tae Kim. 2012. Hybrid recommender system with temporal information. In *ICoin*. IEEE, 421–425.
- [96] Saúl Vargas, Linas Baltrunas, Alexandros Karatzoglou, and Pablo Castells. 2014. Coverage, redundancy and size-awareness in genre diversity for recommender systems. In *RecSys*. 209–216.
- [97] Soroush Vosoughi, Deb Roy, and Sinan Aral. 2018. The spread of true and false news online. *Science* 359, 6380 (2018), 1146–1151.
- [98] Guoping Wang and Chee-Yong Chan. 2013. Multi-query optimization in mapreduce framework. *PVLDB* 7, 3 (2013), 145–156.
- [99] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. 2019. KGAT: Knowledge Graph Attention Network for Recommendation. In *SIGKDD*. ACM, 950–958.
- [100] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *SIGIR*. ACM, 165–174.
- [101] Xinxin Wang and Ye Wang. 2014. Improving content-based and hybrid music recommendation using deep learning. In *MM*. 627–636.
- [102] Zihong Wang, Yingxia Shao, Jiyuan He, Jinbao Liu, Shitao Xiao, Tao Feng, and Ming Liu. 2023. Diversity-aware deep ranking network for recommendation. In *CIKM*. 2564–2573.
- [103] Qiong Wu, Yong Liu, Chunyan Miao, Binqiang Zhao, Yin Zhao, and Lu Guan. 2019. PD-GAN: Adversarial learning for personalized diversity-promoting recommendation. In *IJCAI*, Vol. 19. 3870–3876.
- [104] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. 2022. Graph neural networks in recommender systems: a survey. *Comput. Surveys* 55, 5 (2022), 1–37.
- [105] Dong Xin, Jiawei Han, and Kevin C Chang. 2007. Progressive and selective merge: Computing top-k with ad-hoc ranking functions. In *SIGMOD*. 103–114.
- [106] Zezhong Xu, Yincen Qu, Wen Zhang, Lei Liang, and Huajun Chen. 2024. InBox: Recommendation with Knowledge Graph using Interest Box Embedding. *PVLDB* 17, 13 (2024), 4641–4654.
- [107] Xifeng Yan, Philip S Yu, and Jiawei Han. 2005. Substructure similarity search in graph databases. In *SIGMOD*. 766–777.
- [108] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for Web-scale recommender systems. In *SIGKDD*. 974–983.
- [109] Albert Yu, Pankaj K Agarwal, and Jun Yang. 2012. Processing a large number of continuous preference top-k queries. In *SIGMOD*. 397–408.
- [110] Min Zhang, Dongxin Zhang, Yin Zhang, Kristin Yeager, and Taylor N Fields. 2023. An exploratory study of Twitter metrics for measuring user influence. *Journal of Informetrics* 17, 4 (2023), 101454.
- [111] Zhijie Zhang, Yujie Lu, Weiguo Zheng, and Xuemin Lin. 2024. A comprehensive survey and experimental study of subgraph matching: Trends, unbiasedness, and interaction. *SIGMOD* 2, 1 (2024), 1–29.
- [112] Yuying Zhao, Yu Wang, Yunchao Liu, Xueqi Cheng, Charu C Aggarwal, and Tyler Derr. 2025. Fairness and diversity in recommender systems: A survey. *TIST* 16, 1 (2025), 1–28.
- [113] Lei Zheng, Vahid Noroozi, and Philip S Yu. 2017. Joint deep modeling of users and items using reviews for recommendation. In *WSDM*. 425–434.
- [114] Yanping Zheng, Zhewei Wei, Frank de Hoog, Xu Chen, Hongteng Xu, Yuhang Ye, and Jiadeng Huang. 2025. Lighter-X: An Efficient and Plug-and-Play Strategy for Graph-Based Recommendation through Decoupled Propagation. *PVLDB* 18, 11 (2025), 3721–3729.
- [115] Ziwei Zhu, Yun He, Xing Zhao, and James Caverlee. 2021. Popularity bias in dynamic recommendation. In *SIGKDD*. 2439–2449.

APPENDIX

This appendix is organized as follows.

- Appendix A: Details about Rule Discovery Algorithm
- Appendix B: Details of CTR Algorithm
- Appendix C: Details of TopK Algorithm
- Appendix D: Details about Experiments

Appendix A: Details about Rule Discovery Algorithm.

This section presents the details of rule discovery algorithm OMRLearner, which mines OMRs by extending the learning-based TIE discovery algorithm [28]. We start with the criteria.

Criteria. We use *support* and *confidence* to measure the quality of OMRs. Given an OMR $\varphi = Q[x_0, y_0, w_0](X \rightarrow \text{Rec}_d(x_0, w_0))$ and a graph G , denote by $\text{Rec}_d(G, \varphi)$ the set of all alternatives predicted by φ . That is, $(u, w) \in \text{Rec}_d(G, \varphi)$ if (a) $u = h(x_0)$, $v = h(y_0)$ and $w = h(w_0)$ for some match h of Q in G , and (b) $h \models X$.

Support. The support of an OMR φ measures how often φ can be applied. It is assessed w.r.t. training data $\mathcal{T}$. Here $\mathcal{T}$ is a set of user-item pairs within a graph G , which are real-life interactions and can serve as ground truth. Then the support of φ over a graph G w.r.t. $\mathcal{T}$, denoted by $\text{supp}(\varphi, G, \mathcal{T})$, is defined as

$$\text{supp}(\varphi, G, \mathcal{T}) = |\text{Rec}_d(G, \varphi) \cap \{(u, w) \mid (u, \text{clicked}, w) \in \mathcal{T}\}|.$$

This metric is *anti-monotonic* w.r.t. a partial order $\preceq$ on the domain of OMRs. Given two OMRs $\varphi(Q, X)$ and $\varphi'(Q', X')$, $\varphi \preceq \varphi'$ if (a) Q is a subgraph of Q' and (b) all the predicates in X are also included in X' . One can verify that $\text{supp}(\varphi', G, \mathcal{T}) \leq \text{supp}(\varphi, G, \mathcal{T})$ if $\varphi \preceq \varphi'$.

Confidence. Given an OMR φ , its *confidence* over a graph G w.r.t. training data $\mathcal{T}$, denoted as $\text{conf}(\varphi, G, \mathcal{T})$, is defined as

$$\text{conf}(\varphi, G, \mathcal{T}) = \frac{\text{supp}(\varphi, G, \mathcal{T})}{|\text{Rec}_d(G, \varphi) \cap \{(u, w) \mid (u, \tau, w) \in \mathcal{T}\}|},$$

where τ is clicked or non-clicked. The confidence measures the association strength between the precondition of φ and the predictions made by φ , i.e., the “precision” of φ by referencing ground truth in $\mathcal{T}$. An OMR with high confidence helps us increase the diversity.

OMR discovery. We now formalize the OMR discovery problem.

- **Input:** A graph G , the training data $\mathcal{T}$ in G , a set Σ_{TIE} of TIEs, $\mathcal{M}_{\text{rec}}$, $\mathcal{M}_{\text{dis}}$, $\mathcal{M}_{\text{aggr}}$, a support threshold r_1 , a confidence threshold r_2 , and a positive integer α .
- **Output:** A set Σ_{OMR} of OMRs such that for each $\varphi \in \Sigma_{\text{OMR}}$, $\text{supp}(\varphi, G, \mathcal{T}) \geq r_1$, $\text{conf}(\varphi, G, \mathcal{T}) \geq r_2$, and the pattern of φ has at most α variables.

We use $\text{Req}(r_1, r_2, \alpha)$ to represent an OMR discovery requirement. The problem aims to find a set of OMRs that conform to $\text{Req}(r_1, r_2, \alpha)$, where α controls the cost of the discovery process.

Intuition. To avoid mining OMRs from scratch, we leverage pre-mined TIEs as the basis for OMR discovery. Specifically, given a pre-mined TIE $\varphi_T(Q_T, X_T)$ in Σ_{TIE} , OMRLearner derives a set of candidate patterns for OMRs by preserving the pattern Q_T of φ_T at the pivot variables (x_0, y_0) and extending it via (a) a *bridge path* to an alternative variable w_0 and (b) a *star pattern* centered at w_0 .

However, enumerating such extensions over the entire graph is computationally expensive, since it requires exploring a large number of instance-level paths and star expansions around each match

Input: graph G , a set Σ_{TIE} of TIEs, $\mathcal{M}_{\text{rec}}$, $\mathcal{M}_{\text{dis}}$, $\mathcal{M}_{\text{aggr}}$, and $\text{Req}(r_1, r_2, \alpha)$
Output: a set Σ_{OMR} of OMRs

1. $\Sigma_{\text{OMR}} := \emptyset$; $\mathcal{S} := \text{buildSchema}(G)$;
2. **for each** TIE $\varphi_T \in \Sigma_{\text{TIE}}$ **do**
3. $Q := \text{GenPatterns}(Q_T, \mathcal{S}, \alpha)$;
4. **for each** $Q \in Q$ **do**
5. $\Theta_{\text{dis}}, \Theta_{\text{aggr}} := \text{SelectThresholds}(Q, \varphi_T, G)$;
6. $\Delta := \text{GenPredicates}(Q, X_T, \Theta_{\text{dis}}, \Theta_{\text{aggr}})$;
7. **for each** $(\delta_d, \delta_a) \in \Theta_{\text{dis}} \times \Theta_{\text{aggr}}$ **do**
8. $X := X_T \cup \{\mathcal{M}_{\text{rec}}(x_0, y_0), \mathcal{M}_{\text{dis}}(y_0, w_0) \geq \delta_d, \mathcal{M}_{\text{aggr}}(w_0) \geq \delta_a\}$;
9. $\varphi := Q[x_0, y_0, w_0](X \rightarrow \text{Rec}_d(x_0, w_0))$;
10. **if** φ is r_1 -frequent and r_2 -confident **then**
11. $\Sigma_{\text{OMR}} := \Sigma_{\text{OMR}} \cup \{\varphi\}$;
12. **elseif** φ is r_1 -frequent but not r_2 -confident **then**
13. $\Sigma'_{\text{OMR}} := \text{Expand}(G, Q, X, \Delta)$; $\Sigma_{\text{OMR}} := \Sigma_{\text{OMR}} \cup \Sigma'_{\text{OMR}}$;
14. **Return** Σ_{OMR} ;

Figure 10: OMRLearner.

of pivots (x_0, y_0) , and many of which are structurally redundant and differ only in their concrete instances. To reduce the search space, OMRLearner performs the enumeration over a *graph schema*, i.e., a compact type-level abstraction of the graph that merges structurally identical instance-level extensions.

In addition, unlike TIEs, the precondition of an OMR incorporates dissimilarity and popularity constraints, captured by $\mathcal{M}_{\text{dis}}$ and $\mathcal{M}_{\text{aggr}}$, respectively. Instead of assuming these thresholds are user-specified or exhaustively searching over them, OMRLearner learns them by extracting quantile-based threshold candidates from the empirical distributions of $\mathcal{M}_{\text{dis}}$ and $\mathcal{M}_{\text{aggr}}$ over the raw matches of candidate patterns for OMRs. During predicate generation, it further exploits the anti-monotonicity of support to prune candidates whose support already falls below the given threshold, thus avoiding exhaustive exploration of predicate combinations.

Overview. Given a set Σ_{TIE} of pre-mined TIEs, OMRLearner processes each TIE φ_T in Σ_{TIE} to derive OMRs in two steps.

- (1) **Pattern generation.** Starting from the TIE pattern Q_T of φ_T , OMRLearner derives a set Q of candidate OMR patterns by preserving Q_T at the pivot variables (x_0, y_0) and extending it with (a) a bridge path ρ_B connecting the anchor variable y_0 to a new alternative variable w_0 of item type, and (b) a star pattern Q_w centered at w_0 , i.e., $Q = \{Q^{(i)}\}$, where $Q^{(i)} = Q_T \cup \rho_B^{(i)} \cup Q_w^{(i)}$. To do this, it enumerates those extensions over the graph schema $\mathcal{S}$, rather than directly over the input graph.
- (2) **Predicate generation.** For each candidate pattern in Q , it constructs a set of preconditions by extending X_T of the TIE φ_T with dissimilarity and popularity constraints, whose thresholds are selected using a *quantile-based selection* strategy. It then evaluates support and confidence, and outputs the set Σ_{OMR} of candidate OMRs satisfying support and confidence thresholds.

Algorithm. As outlined in Figure 10, OMRLearner takes as input a graph G , a set Σ_{TIE} of pre-mined TIEs, models $\mathcal{M}_{\text{rec}}$, $\mathcal{M}_{\text{dis}}$, $\mathcal{M}_{\text{aggr}}$, and $\text{Req}(r_1, r_2, \alpha)$. It outputs a set Σ_{OMR} of discovered OMRs.

The algorithm first initializes Σ_{OMR} to the empty set and constructs the graph schema $\mathcal{S}$ from G via procedure `buildSchema` (Line 1). It then processes each TIE $\varphi_T \in \Sigma_{\text{TIE}}$ one by one (Lines 2-13). For the current TIE φ_T , it generates a set Q of candidate OMR

patterns from the TIE pattern Q_T via procedure `GenPatterns`, guided by the schema S and bounded by α (Line 3). For each candidate pattern $Q \in \mathcal{Q}$, it selects two sets of threshold candidates, Θ_{dis} and Θ_{aggr} , for the dissimilarity and popularity measure, respectively, via procedure `SelectThresholds` (Line 5). It then invokes procedure `GenPredicates` to generate a set Δ of candidate predicates for the current pattern Q , to be used in subsequent rule expansion (Line 6).

Next, it starts mining preconditions for Q by enumerating all threshold pairs $(\delta_d, \delta_a) \in \Theta_{\text{dis}} \times \Theta_{\text{aggr}}$ (Lines 7-13). Specifically, for each pair (δ_d, δ_a) , it first constructs a base precondition X by extending the TIE precondition X_T with the recommendation predicate $\text{Rec}(x_0, y_0)$, the dissimilarity constraint $\mathcal{M}_{\text{dis}}(y_0, w_0) \geq \delta_d$, and the popularity constraint $\mathcal{M}_{\text{aggr}}(w_0) \geq \delta_a$ (Line 8). Based on current Q and X , it forms a candidate OMR φ (Line 9). If φ is both r_1 -frequent, *i.e.*, its support is no smaller than the support threshold r_1 , and r_2 -confident, *i.e.*, its confidence is greater than the confidence threshold r_2 , the algorithm adds φ to Σ_{OMR} (Lines 10-11). Otherwise, if φ is r_1 -frequent but not r_2 -confident, the algorithm refines it via procedure `Expand`, which appends predicates from Δ in a levelwise manner to derive more specific OMRs satisfying the support and confidence requirements. All expanded OMRs returned by this procedure are then added to Σ_{OMR} (Lines 12-13). After all TIEs have been explored, the algorithm returns the final set Σ_{OMR} (Line 14).

Below we present the details of procedures used in `OMRLearner`.

Procedure `buildSchema`. Given a data graph $G(V, E)$, procedure `buildSchema` constructs its graph schema $\mathcal{S}(V_S, E_S)$, where each node $A \in V_S$ represents a vertex type occurring in G , and each typed edge $(A, l, B) \in E_S$ indicates that there exists at least one edge $(v, l, v') \in E$ in G such that $L(v) = A$ and $L(v') = B$.

To build the schema, this procedure scans all vertices and edges in G and identifies the set of vertex types appearing in G . For each vertex type A , it creates a schema node in $\mathcal{S}$. Then, for each edge $(u, l, v) \in E$, where $L(u) = A$ and $L(v) = B$, it inserts a schema edge (A, l, B) into E_S . Duplicate schema edges are merged.

Procedure `GenPatterns`. Given a TIE pattern Q_T , graph schema $\mathcal{S}$ and a size bound α , this procedure generates a set $\mathcal{Q}$ of candidate OMR patterns by extending Q_T with a bridge path and an alternative-centered star pattern.

It first generates *bridge paths* from the schema $\mathcal{S}$. Specifically, starting from the type of the anchor variable y_0 in Q_T , it enumerates schema paths of length at most L that end at the item type, so that the endpoint can be instantiated as the alternative item variable w_0 . Each schema path induces a bridge path ρ_B by introducing fresh variables for intermediate schema nodes. For star pattern Q_w , it considers all schema edges incident to the type of w_0 , and treats each as a branch rooted at w_0 . By enumerating those branches, it generates a set of star pattern Q_w pivoted at w_0 describing the local neighborhood of the alternative variable w_0 .

For each pair (ρ_B, Q_w) , it forms a candidate OMR pattern Q as $Q_T \cup \rho_B \cup Q_w$. It prunes those candidates whose size exceed α .

Procedure `SelectThresholds`. Given a candidate pattern Q , a TIE φ_T and a graph G , this procedure selects two sets Θ_{dis} and Θ_{aggr} of candidate thresholds for $\mathcal{M}_{\text{dis}}$ and $\mathcal{M}_{\text{aggr}}$, respectively.

To do this, it first enumerates the matches of Q in G that satisfy the TIE precondition X_T . For each match h , it computes the pair

of scores by $\mathcal{M}_{\text{dis}}$ and $\mathcal{M}_{\text{aggr}}$, *i.e.*, $d(h) = \mathcal{M}_{\text{dis}}(h(y_0), h(w_0))$ and $a(h) = \mathcal{M}_{\text{aggr}}(h(w_0))$. This yields two empirical distributions, one for dissimilarity and the other for the popularity.

Since the observed values in these distributions may be numerous, directly using all distinct values as thresholds would lead to an unnecessarily large search space. Thus, this procedure applies quantile-based strategy. That is, it sorts each distribution, extracts cutoffs as predefined quantile levels, removes duplicates, and returns the resulting ordered sets Θ_{dis} and Θ_{aggr} .

Procedure `GenPredicates`. Given a candidate pattern Q , the TIE precondition X_T , and the threshold candidate sets Θ_{dis} and Θ_{aggr} , this procedure generates a set Δ of candidate predicates.

It first instantiates numeric predicates associated with the alternative item, and then derives additional predicates from the newly introduced part of Q , *i.e.*, the bridge path ρ_B and star pattern Q_w , by using the predicate construction method in [28]. To control the search space, this procedure removes predicates already implied by the TIE precondition X_T , inconsistent with the schema of Q , or redundant under equivalent variable roles.

Procedure `Expand`. Given G , a candidate pattern Q , a base precondition X , and a predicate set Δ , this procedure returns a set Σ'_{OMR} of OMRs that satisfies the support and confidence thresholds.

Specifically, it performs a levelwise manner over refinements of the form of $X' = X \cup \Gamma$ where $\Gamma \subseteq \Delta$. For each refinement, it evaluates the corresponding OMR against the support and confidence thresholds. A rule is returned if it satisfies both thresholds. A candidate violating the support threshold is pruned by anti-monotonicity. The procedure terminates when no promising refinements remain.

Levelwise manner. The expansion process starts with $X_0 = \{X\}$, where X includes ML predicates $\mathcal{M}_{\text{rec}}(x_0, y_0)$, $\mathcal{M}_{\text{dis}}(y_0, w_0) \geq \delta_d$ and $\mathcal{M}_{\text{aggr}}(w_0) \geq \delta_a$ as in Figure 10.

For each expansion to generate X_i , it picks a predicate p from Δ and extends X to $X \wedge p$; it checks the derived OMR $\varphi = Q[x_0, y_0, w_0](X \wedge p \rightarrow \text{Rec}_d(x_0, w_0))$ w.r.t. $\text{Req}(r_1, r_2, \alpha)$. If $\text{supp}(\varphi, G, \mathcal{T}) \geq r_1$ and $\text{conf}(\varphi, G, \mathcal{T}) \geq r_2$, then it adds $X \wedge p$ to $\mathbb{X}$. Otherwise if only $\text{supp}(\varphi, G, \mathcal{T}) \geq r_1$ holds, then it adds $X \wedge p$ to X_{i+1} for further exploration. Note that $X \wedge p$ is not picked if $\text{supp}(\varphi, G, \mathcal{T}) < r_1$, due to the anti-monotonicity of the support. When no X_i can be further extended, the process terminates.

Complexity analysis. Denote by (a) $|\mathcal{Q}|_{\text{max}}$ the maximum number of candidate OMRs patterns per TIE, (b) n_{mat} the maximum number of matches for a candidate pattern in G , (c) c_1 the unit cost to enumerate matches for a candidate pattern, (d) $|\Delta|_{\text{max}}$ the maximum number of candidate predicates generated for a pattern, and (e) c_2 the unit cost to expand predicates for a candidate pattern. In total, `OMRLearner` takes $O(|V| + |E| + |\Sigma_{\text{TIE}}| |\mathcal{Q}|_{\text{max}} (c_1 + n_{\text{mat}} \log n_{\text{mat}} + |\Delta|_{\text{max}} + c_2))$ time cost; since (1) it takes $O(|V| + |E|)$ to build the graph schema $\mathcal{S}$; (2) it subsequently processes $|\Sigma_{\text{TIE}}|$ TIEs, and for each TIE, it generates $O(|\mathcal{Q}|_{\text{max}})$ candidate patterns over $\mathcal{S}$; (3) for each candidate pattern, it enumerates its matches in $O(c_1)$ time sorts two empirical distributions in $O(n_{\text{mat}} \log n_{\text{mat}})$ time, and generates $O(|\Delta|_{\text{max}})$ predicates; and (4) it expands predicates in $O(c_2)$ time.

Appendix B: Details of CTR Algorithm

This section presents more details of the CTR algorithm, includ-

ing the ranking score and each procedure in CTR.

Appendix B-1: Ranking Score. Given an OMR φ , the contribution $\Delta_\varphi(u, v, w)$ of dissimilarity given by φ is defined as:

$$\Delta_\varphi(u, v, w) = \sum_{z \in \bar{x} \cup \bar{y}} s(h(z)) = \sum_{z \in \bar{x} \cup \bar{y}} \sum_{A \text{ in } X} \mathbb{I}_{z.A} \cdot \beta_{h(z).A},$$

where $z \in \bar{x} \cup \bar{y}$ is a variable in pattern Q of φ , $h(z)$ is a matched vertex at z by a match h of Q such that $h \models X$, $s(h(z))$ is the score of $h(z)$, $\mathbb{I}_{z.A}$ is a Boolean function that checks whether $z.A$ appears in X of φ , and $\beta_{h(z).A}$ is the feature (i.e., attribute value) importance of $h(z).A$. Intuitively, $\beta_{h(z).A}$ distinguishes the priority of actual attribute values in different h , e.g., a poster with a larger number of fans is more desirable (i.e., an opinion leader in social networks).

Appendix B-2: Procedure InitCand. Given user u and rule group H , this procedure initializes a shared candidate map

$$C^H = \{C^H(z) \subseteq V \mid z \in \text{Var}(Q^H)\}.$$

It sets $C^H(x_0) = \{u\}$ and, for each $z \neq x_0$, defines

$$C^H(z) = \{v \in V \mid v \text{ satisfies the unary constraints of } z\}.$$

A vertex v is included in $C^H(z)$ if $L_G(v) = L_{Q^H}(z)$, $\deg_G(v) \geq \deg_{Q^H}(z)$, and moreover, v satisfies all unary predicates on z in X^H (e.g., $z.A \oplus c$, ML predicates such as $M_{\text{rec}}(x_0, y_0) \otimes \delta$, $M_{\text{dis}}(y_0, w_0) \otimes \delta$, and popularity predicates $M_{\text{aggr}}(w_0) \otimes \delta$).

Appendix B-3: Procedure constructGroup. Given a user u , a set Σ_{OMR} of OMRs and a TIE $\varphi_T = Q_T[x_0, y_0](X_T \rightarrow \text{Rec}(x_0, y_0)) \in \Sigma_{\text{TIE}}$, this procedure constructs (a) a *rule group* $H = \mathcal{H}(\varphi_T)$ tailored for φ_T and induces (b) a *joint pattern* Q^H for this group H , followed by building (c) a *rooted DAG* Q_D^H of this joint pattern Q^H , as follows.

(a) *Rule group.* For each TIE φ_T , we construct a rule group

$$\mathcal{H}(\varphi_T) = \{\varphi_T\} \cup \Sigma_{\text{OMR}}^{\varphi_T},$$

where $\Sigma_{\text{OMR}}^{\varphi_T} \subseteq \Sigma_{\text{OMR}}$ consists of OMRs whose constraints on the pivot variables (x_0, y_0) are compatible with φ_T . An OMR $\varphi = Q_\varphi[x_0, y_0, w_0](X_\varphi \rightarrow \text{Rec}_d(x_0, w_0))$ is included in $\Sigma_{\text{OMR}}^{\varphi_T}$ if its constraints on x_0 are satisfied by user u , i.e., $L_G(u) = L_{Q_\varphi}(x_0)$, $\deg_G(u) \geq \deg_{Q_\varphi}(x_0)$, and u satisfies all constant predicates on x_0 in X_φ , where $\deg$ denotes the degree in G and Q_φ , respectively.

(b) *Joint pattern.* Given a rule group H , we construct a *joint pattern*

$$Q^H = \bigcup_{\varphi \in H} Q_\varphi,$$

where Q^H consists of variable set $V(Q^H)$ and edge set $E(Q^H)$. It encodes both the TIE topology for anchor selection and the OMR topology for generating alternatives for the same pivot pair (x_0, y_0) .

(c) *Rooted DAG.* We construct a rooted DAG Q_D^H from Q^H by treating the user variable x_0 as the root. We perform a BFS traversal from x_0 and orient edges from higher to lower levels, ensuring that Q_D^H is acyclic and that only x_0 has no incoming edges.

To improve pruning efficiency, variables at the same level are ordered by selectivity. Specifically, we group variables by label and sort the groups in ascending order of their frequency in G ; within each group, variables are ordered by decreasing degree.

Appendix B-4: Procedure localRefinement. Given the current candidate map C^H and a rule $\varphi(Q, X) \in H$, this procedure refines C^H to F_φ by enforcing (a) topological consistency with Q and (b) the constraints in X . It performs a forward pass and a backward pass over the DAG Q_D^H .

- *Forward pass.* For each variable $z \in V_Q$, a vertex v remains in $F_\varphi(z)$ iff (a) $v \in C^H(z)$; (b) for each child z_c of z in Q_D^H , there is an adjacent $v_c \in F_\varphi(z_c)$; and (c) for each variable predicate $z.A \oplus z_p.B$ in X , there is $v_p \in F_\varphi(z_p)$ such that $v.A \oplus v_p.B$ holds.
- *Backward pass.* Traversing the reverse Q_D^{-1} of graph Q_D^H , a vertex v remains in $F_\varphi(z)$ iff $v \in C^H(z)$ and, for every parent z_p of z , there exists an adjacent vertex $v_p \in F_\varphi(z_p)$.

Unlike prior dynamic-programming approaches [47], we do not iterate these passes for a single rule. Instead, refinement is coordinated at the group level, allowing convergence to be determined over H as a whole and avoiding redundant per-rule iteration.

Appendix B-5: Procedure updateScore. When multiple OMRs (or matches of the same OMR) produce the same alternative w for an anchor v , updateScore aggregates their contributions to the ranking score $s(u, v, w)$ incrementally, avoiding recomputation from scratch.

Both the topic proximity $\text{sim}_T(v, w)$ and the acceptability term $s_{\text{acc}}(u, w)$, which depend only on $\mathcal{M}(u, w)$ and $\mathcal{M}_{\text{aggr}}(w)$, are fixed once computed. Only the content dissimilarity component $s_{\text{dis}}(u, v, w, \varphi)$ varies across OMRs. Thus, $s(u, v, w)$ is updated by adding the incremental contribution of the current φ .

Specifically, for each $(v, w) \in F_\varphi(y_0, w_0)$, if w already appears in A_v , we increase $s(u, v, w)$ by $\lambda \cdot \gamma \cdot \Delta_\varphi(u, v, w)$; otherwise, we compute $s(u, v, w) = (1 - \lambda) s_{\text{acc}}(u, w) + \lambda \cdot \text{sim}_T(v, w) \cdot \frac{1}{|\Sigma|} \cdot s_{\text{dis}}(u, v, w, \varphi)$. The correctness of the process is established by Lemma 1.

Appendix B-6: Procedure groupRefinement. Rather than requiring all rules to hold simultaneously, procedure groupRefinement adopts *existential rule support*: a candidate vertex for variable $z \in Q^H$ is retained if it is supported by at least one rule in H via a valid local refinement. This matches the witness-based semantics of TIEs and OMRs. Formally, given local refinements F_φ for each $\varphi \in H$, the updated candidate map is

$$F_H = \bigcup_{\varphi \in H} F_\varphi(C^H).$$

Starting from C^H , the algorithm iteratively applies

$$F_H^{i+1} := \text{groupRefinement}(F_H^i)$$

until convergence. Rules in H may be applied in any order. Once F_H reaches a fixpoint, we extract anchors from $F_H^\infty(y_0)$ that satisfy the TIE precondition X_T . For each anchor v , we choose the highest-scoring alternatives from $F_H^\infty(w_0)$ supported by some rule $\varphi \in H$.

Appendix B-7: Proof of Lemma 1

Proof: Consider that a set Σ^w of OMRs in Σ that are applicable at w . Note that these OMRs are processed subsequently in our CTR algorithm. For the first time that w is discovered by an OMR φ in Σ^w , i.e., $w \notin A_v$, its score is computed as follows:

$$s_1 = (1 - \lambda) \cdot s_{\text{acc}}(u, w) + \lambda \cdot \text{sim}_T(v, w) \cdot \frac{1}{|\Sigma|} \cdot s_{\text{dis}}(u, v, w, \varphi).$$

Let s_i denote the addition for i -th OMR φ_i in Σ^w where $i > 1$. Then,

$$\begin{aligned} s_1 + \sum_{i=2}^{|\Sigma^w|} s_i &= s_1 + \sum_{i=2}^{|\Sigma^w|} \lambda \cdot \text{sim}_T(v, w) \cdot \frac{1}{|\Sigma|} \cdot s_{\text{dis}}(u, v, w, \varphi_i) \\ &= s_1 + \lambda \cdot \frac{1}{|\Sigma|} \cdot \text{sim}_T(v, w) \cdot \sum_{\varphi \in \Sigma^w \setminus \varphi_1} s_{\text{dis}}(u, v, w, \varphi) \\ &= (1 - \lambda) \cdot s_{\text{acc}}(u, w) + \lambda \cdot \frac{1}{|\Sigma|} \cdot \text{sim}_T(v, w) \cdot \sum_{\varphi \in \Sigma^w} s_{\text{dis}}(u, v, w, \varphi) \\ &= s(u, v, w). \end{aligned}$$

This completes the proof. $\square$

Appendix B-8: Proof of Lemma 2

Proof: (1) We first prove that each local refinement operator F_φ is monotone. For a fixed rule $\varphi \in H$, and $C^H \sqsubseteq C'^H$ such that $C^H(z) \subseteq C'^H(z)$ for all z , we prove that $F_\varphi(C^H) \sqsubseteq F_\varphi(C'^H)$. When the context is clear, we use C^H and C interchangeably.

Consider any variable $z \in \text{Var}(Q^H)$ and any vertex $v \in F_\varphi(C)(z)$. If $z \notin \text{Var}(Q_\varphi)$, then by construction of localRefinement, the candidate map remains unchanged, i.e., $F_\varphi(C)(z) = C(z)$ and $F_\varphi(C')(z) = C'(z)$. Since $C(z) \subseteq C'(z)$, we have $v \in F_\varphi(C')(z)$. Now, suppose $z \in \text{Var}(Q_\varphi)$. By the construction of localRefinement, $v \in F_\varphi(C)(z)$ means that v satisfies all constraints enforced by X in φ in the forward and backward passes w.r.t. the candidate map C . Since $C \sqsubseteq C'$, all candidate sets available under C are also available under C' . Thus, each vertex used to justify $v \in F_\varphi(C)(z)$ is still present in the corresponding set under C' . Hence, $v \in F_\varphi(C')(z)$. Since z and v are arbitrary, this proves that $F_\varphi(C) \sqsubseteq F_\varphi(C')$.

(2) We prove that the group refinement operator F_H is also monotone. Since union is monotone w.r.t. the set inclusion and each F_φ is monotone, we have the following:

$$F_H(C) = \bigcup_{\varphi \in H} F_\varphi(C) \sqsubseteq \bigcup_{\varphi \in H} F_\varphi(C') = F_H(C').$$

Thus, F_H is monotone.

(3) We prove that the group refinement can reach a unique fixpoint. First of all, since each local refinement $F_\varphi(C) \sqsubseteq C$, we have, for each $z \in \text{Var}(Q^H)$, $F_H(C)(z) = \bigcup_{\varphi \in H} F_\varphi(C)(z) \subseteq C(z)$. Thus, $F_H(C) \sqsubseteq C$, indicating that F_H is also contractive.

Then, starting from the initial candidate map, denoted by C , we define the updated candidate map as $F_H^{i+1} = F_H^i(C)$. By contractiveness, $F_H^{i+1} \sqsubseteq F_H^i$ for all $i \geq 0$. There exists some $k \geq 0$ such that $F_H^{k+1} = F_H^k$. Let $F_H^\infty = F_H^k$, F_H^∞ is a fixpoint. The proof completes. $\square$

Appendix B-9: Proof of Theorem 3

Proof: (1) We first prove the soundness. Let $(v, w) \in \text{Cpl}(u, H)$. By fixpoint extraction, there exists some $\varphi \in H \setminus \{\varphi_T\}$ and a witness match h_φ satisfying X_φ such that $h_\varphi(x_0) = u$, $h_\varphi(y_0) = v$, and $h_\varphi(w_0) = w$. Since v is output as an anchor, there also exists a witness h_T of φ_T such that $h_T(x_0) = u$ and $h_T(y_0) = v$. Hence, the sequential algorithm first outputs v in the TIE stage and then outputs w for v in the OMR stage, so $(v, w) \in \text{Seq}(u, H)$.

(2) We then prove the completeness. Let $(v, w) \in \text{Seq}(u, H)$. In this case, there exist a TIE witness h_T and an OMR witness h_φ from some $\varphi \in H \setminus \{\varphi_T\}$ such that $h_T(x_0) = h_\varphi(x_0) = u$, $h_T(y_0) = h_\varphi(y_0) = v$, and $h_\varphi(w_0) = w$. Since φ is compatible with φ_T on the pivot variables, $h = h_T \cup h_\varphi$ is well-defined. Each vertex in h belongs to the initialized candidate map and is supported by valid witnesses for the forward/backward passes and moreover, no iteration of group refinements F_H removes it. Thus, $v \in F_H^\infty(y_0)$ and $w \in F_H^\infty(w_0)$, making $(v, w) \in \text{Cpl}(u, H)$.

Combining both, we have $\text{Seq}(u, H) = \text{Cpl}(u, H)$. $\square$

Appendix B-10: Proof of Theorem 4

Proof: To prove this, we show the stronger inclusion $\mathcal{N}_{\text{Cpl}} \subseteq \mathcal{N}_{\text{Seq}}$. For a fixed OMR $\varphi \in H \setminus \{\varphi_T\}$. Let C_H^i and $C_{\varphi, \text{Seq}}^i$ be the candidate

maps maintained at the i -th iteration by the coupled and sequential algorithm, respectively. We want to prove that for each $i \geq 0$,

$$C_H^i(y_0) \subseteq C_{\varphi, \text{Seq}}^i(y_0), \text{ and } C_H^i(w_0) \subseteq C_{\varphi, \text{Seq}}^i(w_0).$$

We prove the claim by induction. When $i = 0$, it holds since the coupled algorithm initializes the candidate map by performing on the joint pattern Q^H , which contains all constraints of Q_φ and possibly more, and hence is no weaker than the isolated initialization for φ . Assume that it holds at the i -th iteration, i.e.,

$$C_H^i(y_0) \subseteq C_{\varphi, \text{Seq}}^i(y_0), \text{ and } C_H^i(w_0) \subseteq C_{\varphi, \text{Seq}}^i(w_0).$$

We prove that at the $(i + 1)$ -th iteration,

$$C_H^{i+1}(y_0) \subseteq C_{\varphi, \text{Seq}}^{i+1}(y_0), \text{ and } C_H^{i+1}(w_0) \subseteq C_{\varphi, \text{Seq}}^{i+1}(w_0).$$

For the sequential algorithm that refines the candidate $C_{\varphi, \text{Seq}}^i$ by enforcing the constraints of φ , we denote this refinement by F_φ , and for variable y_0 (or w_0), we have

$$C_{\varphi, \text{Seq}}^{i+1}(y_0) = F_\varphi(C_{\varphi, \text{Seq}}^i)(y_0), \text{ and } C_{\varphi, \text{Seq}}^{i+1}(w_0) = F_\varphi(C_{\varphi, \text{Seq}}^i)(w_0).$$

Since $C_H^i(y_0) \subseteq C_{\varphi, \text{Seq}}^i(y_0)$ and F_φ is monotone, we get $F_\varphi(C_H^i)(y_0) \subseteq F_\varphi(C_{\varphi, \text{Seq}}^i)(y_0)$. Similarly, $F_\varphi(C_H^i)(w_0) \subseteq F_\varphi(C_{\varphi, \text{Seq}}^i)(w_0)$.

For our coupled algorithm, it evaluates all rules in H over the shared candidate map. The updated C_H^{i+1} is obtained by combining the local refinement results of the rules in H , including the local refinement induced by φ . In particular, when restricted to the variables of Q_φ , each candidate vertex that survives in C_H^{i+1} must survive the constraints of φ applied to the previous map C_H^i . Thus, for variables y_0 and w_0 , we have the following:

$$C_H^{i+1}(y_0) \subseteq F_\varphi(C_H^i)(y_0), \text{ and } C_H^{i+1}(w_0) \subseteq F_\varphi(C_H^i)(w_0).$$

Combining with the inclusions of the sequential algorithm, it yields

$$C_H^{i+1}(y_0) \subseteq C_{\varphi, \text{Seq}}^{i+1}(y_0), \text{ and } C_H^{i+1}(w_0) \subseteq C_{\varphi, \text{Seq}}^{i+1}(w_0).$$

The induction completes.

Since φ is chosen arbitrarily from $H \setminus \{\varphi_T\}$, taking the union over all OMRs yields $\mathcal{N}_{\text{Cpl}} \subseteq \mathcal{N}_{\text{Seq}}$. Consequently, $|\mathcal{N}_{\text{Cpl}}| \leq |\mathcal{N}_{\text{Seq}}|$. $\square$

Appendix C: Details of TopK Algorithm This section presents more details of the TopK algorithm. In addition, it provides the proofs of lemmas and theorems in Section 5.

Appendix C-1: Batch validation via TIEs aggregation. Applying all TIEs to each fuzzy-range item is costly: (1) Many such items will never enter the top- k anchors, yet validating them wastes computation. (2) A single TIE may validate multiple anchors, so evaluating rules per item repeats the same refinements.

To address this, we adopt a *batch validation* strategy. (1) Items are scanned in descending $\mathcal{M}_{\text{rec}}$ by the order-preservation property in Lemma 5. (2) TIEs are evaluated on candidate batches rather than individually, avoiding redundant refinements.

Batch validation via TIEs aggregation. To validate items in batches, we maintain a sliding window over the $\mathcal{M}_{\text{rec}}$ -sorted list H_{rec} . In each iteration, we select a contiguous slice V_{sel} of promising anchors, initially the top k positions. For this batch, we aggregate only the TIEs in Σ_{TIE} whose constraints on y_0 are compatible with at least one $v \in V_{\text{sel}}$. Formally, the unified set $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ is defined:

$$\Sigma_{\text{TIE}}^s(V_{\text{sel}}) = \{\varphi_T \in \Sigma_{\text{TIE}} \mid C^{\varphi_T}(y_0) \cap V_{\text{sel}} \neq \emptyset\},$$

where $C^{\varphi_T}(y_0)$ is the initial candidate map at y_0 under φ_T . Intuitively, a TIE φ_T is selected iff there exists an item $v \in V_{\text{sel}}$ such that v satisfies the unary constraints of y_0 in φ_T .

Input: $G, u, \mathcal{M}_{\text{rec}}, \Sigma_{\text{TIE}}, \delta_H$ and k .

Output: a set $K(u)$ of top- k ranked anchor items

```

1.  $H_{\text{rec}} := \text{Sort}(V_{\text{item}}, \mathcal{M}_{\text{rec}})$ ;  $K(u) := \emptyset$ ;  $i_b := 0$ ;  $i_e := k$ ;
2. for each  $v \in H_{\text{rec}}$  do
3.   if  $|K(u)| < k$  and  $\mathcal{M}_{\text{rec}}(u, v) \geq \delta_H$  then
4.      $K(u) := K(u) \cup \{v\}$ ;  $i_b := i_b + 1$ ;
5.   else break;
6. while  $|K(u)| < k$  do
7.    $V_{\text{sel}} := \{H_{\text{rec}}(j) \mid i_b \leq j \leq i_e\}$ ;
8.    $\Sigma_{\text{TIE}}^s(V_{\text{sel}}) := \{\varphi_T \in \Sigma_{\text{TIE}} \mid C^{\varphi_T}(y_0) \cap V_{\text{sel}} \neq \emptyset\}$ ;
9.    $V_{\text{valid}} := \text{Refinement}(V_{\text{sel}}, \Sigma_{\text{TIE}}^s(V_{\text{sel}}, u))$ ;
10.   $K(u) := K(u) \cup V_{\text{valid}}$ ;
11.   $i_b := i_b + 1$ ;  $i_e := i_e + k - |V_{\text{valid}}|$ ;
12. Return  $K(u)$ ;
```

Figure 11: Top- k anchor selection algorithm TopKAnchor

Then, we run coupled refinement over this unified set $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ like in CTR, which simultaneously validates all anchors in V_{sel} . After processing $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$, we discard those in V_{sel} that remain unsupported, and slide the window forward to include the next high- $\mathcal{M}_{\text{rec}}$ items. This process repeats until k anchors are determined.

Appendix C-2: Details of TopKAnchor algorithm. We now present TopKAnchor, the top- k anchor selection algorithm, in Figure 11. Given a graph G , a user u , a set Σ_{TIE} of TIEs, the base strength $\mathcal{M}_{\text{rec}}(u, v)$ for each item v and a constant k , TopKAnchor returns the top- k anchor set $K(u)$.

TopKAnchor first sorts items by descending $\mathcal{M}_{\text{rec}}$ to form H_{rec} , and initializes $K(u)$ and the sliding-window indices i_b and i_e (Line 1). It then scans items whose base score exceeds δ_H and inserts them into $K(u)$ while $|K(u)| < k$ (Lines 2–5). If fewer than k anchors are obtained, the algorithm performs iterative batch validation over the remaining items (Lines 6–11). In each iteration, it selects a candidate batch $V_{\text{sel}} = H_{\text{rec}}[i_b, i_e]$ (Line 7), constructs the unified rule set $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ (Line 8), and invokes Refinement to derive anchors V_{valid} supported by at least one TIE (Line 9). The validated anchors are added to $K(u)$, and the window is advanced by setting $i_b = i_e + 1$ and $i_e \leftarrow i_e + k - |V_{\text{valid}}|$ for the next iteration (Lines 10–11).

Procedure Refinement performs forward and backward passes for each rule in $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ like in CTR until convergence. By initializing the candidate maps $C(x_0) = \{u\}$ and $C(y_0) = V_{\text{sel}}$, it restricts the candidate space, reducing the cost of the refinement passes.

Appendix C-3: Proof of Lemma 3

Proof: We proved this by distinguishing three cases. (a) If $\mathcal{M}_{\text{rec}}(u, v_1) > \mathcal{M}_{\text{rec}}(u, v_2) > \delta_H$, then v_1 is ranked before v_2 . (b) If $\mathcal{M}_{\text{rec}}(u, v_1) > \delta_H > \mathcal{M}_{\text{rec}}(u, v_2)$, then v_1 is selected into $I(u)$ and keeps its origin $\mathcal{M}_{\text{rec}}(u, v_1)$; for v_2 , its value is increased to δ_H , which is smaller than $\mathcal{M}_{\text{rec}}(u, v_1)$, and thus, v_1 is ranked before v_2 . (c) If $\delta_H > \mathcal{M}_{\text{rec}}(u, v_1) > \mathcal{M}_{\text{rec}}(u, v_2)$, both strengths are adjusted to δ_H and remains the same relative order. $\square$

Appendix C-4: Rule-level Upper Bound. Below we present an initialization of $\Delta_{\varphi}^{\text{max}}$ for φ in the offline phase.

$$\Delta_{\varphi}^{\text{max}} = \sum_{z \in \bar{x} \cup \bar{y}} \hat{s}(z),$$

where $\hat{s}(z)$ is the score upper bound of all vertices in G that can be mapped to a variable $z \in \bar{x} \cup \bar{y}$, i.e., $\hat{s}(z) = \max_{h \text{ of } \varphi} s(h(z))$.

Input: $G, u, \Sigma_{\text{OMR}}, \Sigma_{\text{TIE}}, \mathcal{M}_{\text{rec}}, \mathcal{M}_{\text{dis}}, \mathcal{M}_{\text{aggr}}, \lambda, \delta_H$ and k .

Output: a set $K(u)$ of anchor items and a set $K_d(u)$ of alternative items.

```

1.  $\Psi := \emptyset$ ;  $K(u) := \emptyset$ ;  $K_d(u) := \emptyset$ ;
2.  $K(u) := \text{TopKAnchor}(u, \Sigma_{\text{TIE}}, \mathcal{M}_{\text{rec}}, \delta_H, k)$ ;
3.  $H, Q^H, Q_D^H := \text{constructGroup}(u, K(u), \Sigma_{\text{OMR}})$ ;
4.  $C^H := \text{InitCand}(Q^H, u)$ ;  $F_H := C^H$ ;
5. Initialize  $\text{best}_w$  and  $\hat{s}(u, w)$  for each  $w \in F_H(y_0)$ ;
6. for each  $v \in F_H(y_0)$  do
7.   for each  $\varphi \in H$  do
8.      $\text{UB}_{\varphi}(u, v) := \hat{\gamma}(v) \cdot \Delta_{\varphi}^{\text{max}}$ ;
9.   for each  $(v, w) \in F_H(y_0, w_0)$  do
10.     $\hat{s}(u, v, w) := \lambda \cdot \frac{1}{|H|} \sum_{\varphi \in H} \text{UB}_{\varphi}(u, v) + (1 - \lambda) \cdot s_{\text{acc}}(u, w)$ ;
11.    if  $\hat{s}(u, v, w) \geq \hat{s}(u, w)$  then  $\hat{s}(u, w) := \hat{s}(u, v, w)$ ;
12.  while  $F_H$  does not converge do
13.    for each  $\varphi \in H$  do
14.       $F_{\varphi} := \text{localRefinement}(F_H, Q_D^H, \varphi)$ ;
15.    for each  $w \in F_{\varphi}(w_0)$  do
16.       $\tau :=$  the  $k$ -th highest aggregated score in  $\Psi$ ;
17.      if  $\hat{s}(u, w) < \tau$  then Remove  $(w)$  from  $F_{\varphi}(w_0)$ ; continue;
18.      for each  $v$  such that  $v \in F_{\varphi}(y_0)$  and  $(v, w) \in F_{\varphi}(y_0, w_0)$  do
19.         $s(u, v, w) := \text{updateScore}(u, v, w, \mathcal{M}_{\text{dis}}, \mathcal{M}_{\text{aggr}}, \lambda)$ ;
20.        if  $s(u, v, w) > \text{best}_w$  then
21.           $\text{best}_w := s(u, v, w)$ ;
22.          if  $\text{best}_w > \tau$  then Update  $\Psi$  and  $\tau$  by  $w$ ;
23.      Compute (tighter)  $\hat{\gamma}(v)$  and  $\Delta_{\varphi}^{\text{max}}$  based on  $F_{\varphi}$ ;
24.      Update  $\text{UB}_{\varphi}(u, v)$ ;
25.      Update  $\hat{s}(u, v, w')$  for each  $(v, w') \in F_{\varphi}(y_0, w_0)$ ;
26.      Update  $\hat{s}(u, w)$  for each  $w \in F_{\varphi}(w_0)$ ;
27.     $F_H := \text{groupRefinement}(\{F_{\varphi} \mid \varphi \in H\})$ ;
28.  $K_d(u) := \Psi$ ;
29. Return  $K(u)$  and  $K_d(u)$ ;
```

Figure 12: Full version of TopK algorithm.

Intuitively, instead of computing the exact score of $h(z)$ for each variable z , we initialize $\hat{s}(z)$ by the predicates in X since a $h(z)$ must satisfy the predicates constraint on z in X . Initially, $\hat{\gamma}(v)$ is initialized as 1. After one local refinement at φ , for every candidate pair $(v, w) \in F_{\varphi}(y_0, w_0)$, $\hat{\gamma}(v)$ could be updated if some candidates w are discarded, and $\Delta_{\varphi}^{\text{max}}$ is also updated since some candidates $C^H(z)$ on other variable z in φ may be removed. Then, the value of rule-level upper bound $\text{UB}_{\varphi}(u, v)$ can be tightened accordingly.

Appendix C-5: A Full Version of TopK Algorithm. Figure 12 shows a more detailed version of the TopK algorithm. It returns a top- k anchor set $K(u)$ and a top- k alternative set $K_d(u)$.

TopK first initializes an empty top- k heap Ψ and sets the anchor set $K(u)$ and alternative set $K_d(u)$ to be empty (Line 1). It then invokes TopKAnchor to obtain the top- k anchors $K(u)$ (Line 2). Based on $K(u)$, it constructs a rule group H of compatible OMRs from Σ_{OMR} via procedure constructGroup (Line 3). Unlike CTR, TopK restricts H to rules whose constraints on the anchor variable y_0 are satisfied by at least one $v \in K(u)$. It initializes a shared candidate map C^H and sets the group operator $F_H \leftarrow C^H$ (Line 4). Next, for each candidate alternative w , it initializes its aggregated score best_w and global upper bound $\hat{s}(u, w)$ (Line 5). For each $v \in F_H(y_0)$, it initializes the rule-level upper bound $\text{UB}_{\varphi}(u, v)$ for each $\varphi \in H$ (Lines 6–8). Then, for each candidate pair $(v, w) \in F_H(y_0, w_0)$, it initializes the candidate-level bound $\hat{s}(u, v, w)$ by the above definition, and updates the global upper bound $\hat{s}(u, w)$ (Lines 9–11).

The algorithm then performs coupled refinement until convergence (Lines 12-27). For each OMR $\varphi \in H$, it applies procedure `localRefinement` to enforce φ 's preconditions on F_H using Q_D^H (Line 14). It then enumerates each candidate alternative w in current $F_\varphi(w_0)$ (Lines 15-22). For each $w \in F_\varphi(w_0)$, it compares the bound $\hat{s}(u, w)$ with τ : If $\hat{s}(u, w) < \tau$, w is pruned from $F_\varphi(w_0)$ (Lines 16-17). Otherwise, it enumerates all pairs $(v, w) \in F_\varphi(y_0, w_0)$ tied to w , updating $s(u, v, w)$ via `updateScore`; if $s(u, v, w) > \text{best}_w$, it updates best_w ; if the updated aggregated score $\text{best}_w > \tau$, it updates Ψ and threshold τ with w (Lines 18-22). After processing all candidate alternatives, it recomputes the tighter bounds $\hat{y}(v)$ and $\Delta_\varphi^{\max}$ based on F_φ , since candidates of other variables in φ may have been removed after refinement (Line 23). Next, it updates the rule-level bounds $\text{UB}_\varphi(u, v)$, the candidate-level upper bounds $\hat{s}(u, v, w')$ for each $(v, w') \in F_\varphi(y_0, w_0)$, and the global upper bounds $\hat{s}(u, w')$ for each $(w') \in F_\varphi(w_0)$ (Lines 24-26). After processing all rules in H , the algorithm applies `groupRefinement` to update F_H (Line 27).

Once convergence is reached, it returns $K(u)$ and the selected alternatives in Ψ as $K_d(u)$ (Lines 28-29).

Appendix C-6: Proof of Theorem 6

Proof: We prove the theorem in two steps.

(1) We first show that $\hat{s}(u, w)$ is an upper bound of $s(u, w | K(u))$. For a fixed anchor $v \in K(u)$ and any alternative w , by definition, the score $s(u, v, w)$ is determined by the contributions of the OMRs in H together with the accuracy term $s_{\text{acc}}(u, w)$. For each $\varphi \in H$, $\text{UB}_\varphi(u, v)$ is defined as the maximum possible contribution that φ can make to any candidate alternative at (u, v) . Hence, for each w , $\Delta_\varphi(u, v, w) \leq \text{UB}_\varphi(u, v)$. Summing over all OMRs in H , we have $s(u, v, w) \leq \hat{s}(u, v, w)$. Since this holds for each $v \in K(u)$, taking the maximum over all selected anchors gives the following inequalities:

$$s(u, v | K(u)) = \max_{v \in K(u)} s(u, v, w) \leq \max_{v \in K(u)} \hat{s}(u, v, w) = \hat{s}(u, w).$$

Thus, $\hat{s}(u, w)$ is a valid upper bound on the aggregated score of w .

(2) We show that pruning an alternative w when $\hat{s}(u, w) \leq \tau$ is safe. Suppose that at some iteration, for a candidate alternative w , we have $\hat{s}(u, w) < \tau$. By step (1), $\hat{s}(u, w)$ is an upper bound on the true aggregated score of w , i.e.,

$$s(u, w | K(u)) \leq \hat{s}(u, w) < \tau.$$

Hence the true score of w is already strictly below the current top- k threshold, and thus w cannot belong to the current top- k result.

It remains to show that w cannot become feasible later. During refinement, candidates are only removed from the match sets. Thus, all rule-level upper bounds $\text{UB}_\varphi(u, v)$, pairwise bounds $\hat{s}(u, v, w)$, and aggregated bounds $\hat{s}(u, w)$ are monotonically non-increasing. In contrast, the heap threshold τ is monotonically non-decreasing, since the top- k heap only replaces lower-scoring entries with higher-scoring ones. Consequently, once $\hat{s}(u, w) < \tau$ holds at some iteration, it continues to hold in all subsequent iterations. Thus, w can never re-enter the top- k answer, and pruning w is safe. $\square$

Appendix D: Details of Experiments This section gives the details of experimental settings and user studies.

Appendix D-1: Evaluation metrics. Filter Bubble Penetration (FBP) evaluates the proportion of recommended items outside users' echo chamber, defined as:

$$\text{FBP} = \frac{1}{n} \sum_u \frac{|\text{Out}(\mathcal{R}_u)|}{|\mathcal{R}_u|},$$

where $\text{Out}(\mathcal{R}_u)$ is the set of items outside u 's echo chamber, i.e., items whose average similarity score to the items u historically interacted with is below threshold β . This metric follows the exploration metric in [25] that measures the extent to which the recommended items deviate from a user's known interests. The value of threshold β is determined by the average similarity score of each pair of items in the user u 's historical interactions, yielding 0.72, 0.63 and 0.66 on Anime, Steam and Book, respectively.

Appendix D-2: User studies. We conducted two user studies to assess whether OpenMind is better received than existing methods in recommending alternatives with diverse viewpoints. We compared OpenMind with RAPID, Random and SameTopic. This study was conducted on the user-anime review dataset Anime. We provide both qualitative and quantitative evaluations for each user study. The questionnaires for user studies are provided in our artifact link.

(1) Top-1 recommendation. The first user study shows the top-1 recommendation produced by four evaluated methods.

(a) Qualitative evaluation. For a user (ID: "user:Visceras") on Anime, Table 3 shows his profile, favorite anime, and most recently watched anime. Specifically, he is a young male born after 1995. Based on his watched-anime list (i.e., the positive interactions used to train the recommender system), this user appears to prefer anime characterized by *dark tone, psychological tension, supernatural elements, and action-oriented narratives*. Based on this profile and interaction history, a traditional recommender system suggests an anime *Death Note* to this user, which is highly consistent with his historical preferences, as shown in Table 4. Building upon this traditional recommendation, each evaluated method further provides one diversified recommendation for diversity, also reported in Table 4.

Specifically, RAPID recommends an adventure anime *Howl's Moving Castle*, in which a young woman who is cursed into an old body becomes involved with a mysterious wizard and his moving castle. However, this anime differs substantially from *Death Note* in genre, and is also misaligned with the user's preferences; therefore, it is unlikely to be accepted by the user.

Random randomly samples alternatives from the entire anime dataset and recommends *Blue Spring Ride*. However, this recommendation also does not fall within the user's preferences. SameTopic randomly picks an anime with a genre similar to *Death Note*, recommending *Death Note: Rewrite*. Nevertheless, this anime is essentially a recap of the anime already recommended by the traditional recommender, and thus remains within the same perspective.

In contrast, our proposed OpenMind recommends an anime *The Future Diary*, which not only shares similar genres with *Death Note* but also aligns well with the user's preferences. More importantly, it provides a different narrative style, thus broadening the user's horizons and helping to break filter bubbles.

(b) Quantitative evaluation. We adopted four metrics: (1) *Acceptability* (Acc), measuring whether the user would likely accept the recommended items, (2) *Topic Consistency* (TopCon), measuring whether the recommended items remain within the same topic (category) as the anchor, (3) *View Divergence* (ViewD), evaluating whether the recommended items present different viewpoints, and

User profile				Favorite Anime				Latest Watched Anime			
User Name	Gender	Birthdate	Age Group	Anime Name	Year	Genre	Short Description	Anime Name	Year	Genre	Short Description
Visceras	Male	Aug 18, 1997	Post-95s	Clannad After Story	2008	Drama/ romance/ slice of life/ supernatural	A sequel to Clannad that focuses on adulthood, family, loss, and emotional growth.	Terror in Resonance	2014	Psychological thriller/ mystery/ crime	Two teenage terrorists challenge society through a series of attacks and riddles.
				Fate/ Zero	2011	Action/ dark fantasy/ supernatural	A battle-royale story in which mages and heroic spirits fight in the Holy Grail War.	Phantom: Requiem for the Phantom	2009	Psychological/ crime/ action/ drama	A young man is drawn into the underworld after being turned into an assassin.
				Death Note	2006	Psychological thriller/ mystery/ supernatural	A high school student gains a notebook that can kill anyone whose name is written in it.	Tokyo Ghoul	2014	Dark fantasy/ horror/ action/ supernatural	A college student becomes half-ghoul and struggles between human and monster worlds.
				Wolf Children	2012	Drama/ fantasy/ family/ slice of life	A heartfelt film about a mother raising two half-wolf children while protecting their secret.	Fate/ Zero	2011	Action/ dark fantasy/ supernatural	A battle-royale story in which mages and heroic spirits fight in the Holy Grail War.

Table 3: The profile and historical preferences of user (ID:“user:Visceras”) on Anime.

Traditional Recommendation (Anchor)				Diversified Recommendation (Alternative)			
Anime Name	Year	Genre	Short Description	Methods	Anime Name	Year	Short Description
Death Note	2006	Psychological thriller/ mystery/ supernatural	A high school student gains a notebook that can kill anyone whose name is written in it.	RAPID	Howl’s Moving Castle	2004	A young woman cursed into an old body becomes involved with a mysterious wizard and his moving castle.
				Random	Blue Spring Ride	2014	A high school girl reconnects with her first love and navigates youth, friendship, and emotional change.
				SameTopic	Death Note: Rewrite	2007	A recap film that reorganizes the main events of Death Note into a condensed retelling.
				OpenMind	The Future Diary	2011	A boy is forced into a deadly game where participants use diaries that can predict the future

Table 4: Traditional recommendation and the diversified recommendations by evaluated methods.

His Previously-played Games			Traditional top-5 Recommendations to This User		
Game Name	Short Description	Topics (Keywords)	Game Name	Short Description	Topics (Keywords)
Cold Fear	During a severe Arctic storm, a coast-guard veteran boards a mysterious Russian whaler and discovers horrors inside.	horror / investigation/ survival	IT Simulator	Two players cooperate to restore power to an office building, taking the roles of an all-seeing control-room operator and a stealthy repair technician trying to escape.	cooperation/ horror/ stealth
Crysis 2	Aliens invade New York, and the player uses advanced equipment to fight through the city and survive.	science fiction/ shooting/ action			
Deadlight	After a catastrophe, a city is occupied by dangerous infected people; the player runs, jumps, and searches for safe routes.	zombies/ platforming/ survival	Penumbra: Black Plague Gold Edition	The player gathers clues and solves puzzles while surrounded by enemies and psychological terror that threatens the protagonist’s sanity.	psychological horror/ puzzles/ survival
ABZU	The player dives into a colorful underwater world, exploring the ocean and discovering its living creatures.	ocean/ exploration/ relaxing	Montague’s Mount	The player explores a bleak, abandoned Irish island, searching the environment and solving challenging puzzles in a psychological thriller.	psychological thriller/ island exploration/ puzzles
Layers of Fear	The player is a painter exploring a changing Victorian mansion while facing disturbing psychological visions.	psychological horror/ exploration/ story			
Saints Row IV	The player leads a group against an alien invasion, using unusual abilities and weapons across a city.	action/ city exploration/ science fiction	Conarium	The player follows a story about four scientists and explores the strange aftermath of their attempt to challenge the accepted limits of nature.	horror/ scientific mystery/ exploration
Vanquish	This science-fiction action game has the player fight future-tech enemies with a wide range of weapons.	science fiction / shooting/ fast combat			
Agents of Mayhem	The player commands a special-operations team in Seoul against a criminal organization trying to control the world.	action / team-based play/ city adventure	Decay: The Mare	A man entering a treatment institution becomes trapped in an endless nightmare on his first night, and the player explores the setting to find a way out.	psychological horror/ nightmare/ escape

Table 5: The historical preferences of a user on Steam and a traditional top-5 games recommendation list.

(4) *Overall* (All) that considers all the above metrics simultaneously.

We further applied cross-check questions to filter invalid responses. More specifically, we presented two questions for each metric, which asked the participants to select the best recommendation and best two recommendations *w.r.t.* each metric, respectively. If the best-two recommendations do not include the best one, the response was marked as invalid. In total, we collected 76 valid responses, out of 84 responses. For each metric, each respondent selected the best two recommendations.

As Figure 9(l) shows, 61 of the responses indicate that OpenMind performs the best overall. Although RAPID and Random have higher ViewD as both provide across-topic items, they sacrifice the overall performance. Compared with RAPID (resp. Random), OpenMind achieves 39.48% (resp. 55.27%), 67.1% (resp. 75%) and

17.1% (resp. 60.52%) higher Acc, TopCon and All, respectively, verifying the effectiveness of balancing both acceptability and diversity in alternative generation. Compared with SameTopic, OpenMind improves ViewD by 27.63%, showing that the topic preservation alone is insufficient for exposing users to opposing views.

(2) *Top-5 recommendation.* We further examined top-5 recommendation list for a user on Steam, to recommend five games.

(a) *Qualitative evaluation.* From the historical interactions as reported in Table 5, the users mainly enjoy **science-fiction action and shooter games**, with a strong interest in **horror, survival, and exploration**, and occasional openness to relaxing, story-driven experiences. A traditional recommender provides a top-5 list to this user, containing five games, *i.e.*, *IT Simulator*, *Penumbra: Black Plague Gold Edition*, *Montague’s Mount*, *Conarium*, and *Decay: The*

Methods	Game Name	Short Description	Topics (Keywords)
RAPID	Tomb Raider	The player follows Lara as she explores ruins, climbs through environments, solves puzzles, and fights enemies.	exploration/ puzzles/ action
	Life is Strange - Episode 1	The player explores a school and a small town, using the ability to change the past to affect the story.	story/ choices/ time
	Batman: Arkham City - Game of the Year Edition	The player controls Batman, investigating events, moving quietly, and fighting enemies across a city.	hero/ investigation/ action
	Rise of the Tomb Raider	The player follows Lara through wilderness and ancient ruins, solving puzzles and dealing with danger.	exploration/ ruins/ survival
	Shadow of the Tomb Raider: Definitive Edition	Lara tries to stop a Maya apocalypse while exploring ruins and varied environments during her journey.	exploration/ ancient ruins/ action
Random	Sisters Royale: Five Sisters Under Fire	Five sisters compete over the same goal in a fast vertical shooting game that emphasizes aiming and avoiding attacks.	shooting/ arcade/ sisters
	Monster Boy and the Cursed Kingdom	The player adventures through a colorful fantasy world, fights enemies, and uses different forms to solve environmental problems.	fantasy/ adventure/ platforming
	Golf Gang	Players compete in fast mini-golf rounds, alone or online, trying to complete courses quickly.	sports/ racing/ multiplayer
	Teeworlds	Players use weapons in simple multiplayer battles, including team matches, and can also design their own maps.	multiplayer/ battles/ map design
	CHRONO TRIGGER	The player travels across different historical eras, adventures with companions, and tries to save the world.	role-playing/ time travel/ adventure
SameTopic	Half-Life: Blue Shift	The player is a security guard in a research facility who seeks a way out after a disaster and fights enemies.	science fiction/ shooting/ escape
	Half-Life: Caged	The player is trapped in a prison-like setting and must fight, explore, and find a way to escape.	shooting/ exploration/ escape
	Half-Life 2: Episode Two	The player continues through an occupied city and wilderness, working with companions against hostile forces.	science fiction/ adventure/ combat
	Tomb Raider	The player follows Lara as she explores ruins, climbs through environments, solves puzzles, and fights enemies.	exploration/ puzzles/ action
	Redneck Rampage	The player uses weapons against aliens and other enemies in rural and wilderness settings	action/ shooting/ wilderness
OpenMind	Blair Witch	The player investigates a disappearance in a forest while facing strange events that affect perception and fear.	forest/ psychological horror/ investigation
	F.E.A.R.	The player investigates supernatural events while fighting with guns in dark and frightening environments.	horror/ shooting/ supernatural
	ZOMBI	A plague has overwhelmed London; the player searches for scarce supplies and avoids infected people.	zombies/ survival/ resources
	DARQ: Complete Edition	A boy is trapped in a lucid nightmare and must observe the environment, solve puzzles, and find a way out.	dreams/ puzzles/ horror
	State of Decay	After a zombie disaster, the player gathers supplies, helps survivors, and builds a safer base.	zombies/ survivors/ base building

Table 6: Diversified top-5 recommendations by evaluated methods for the given user.

Mare. These recommended games are largely similar in tone and theme. All center on horror and psychological tension, sharing a dark atmosphere built around exploration and puzzle-solving.

Based on this traditional recommendation, each method augments it with diversified recommendations, as reported in Table 6.

Specifically, RAPID reorders the list toward *exploration- and action-oriented games*; however, its top-5 remains dominated by near-duplicate games, *i.e.*, three *Tomb Raider* games plus a comparable action-investigation game (*Batman: Arkham City*), offering only two genuinely distinct viewpoints, and thus limited diversity. Random samples alternatives from the entire dataset, yielding items such as *Golf Gang*, *Teeworlds*, and *Monster Boy and the Cursed Kingdom*; while its list spans more topics (*e.g.*, sports, multiplayer, platforming, role-playing), four of them fall outside the user’s preferences and are unlikely to be accepted. SameTopic selects same-topic games, *e.g.*, *Half-Life: Blue Shift*, *Half-Life: Caged*, and *Half-Life 2: Episode Two*; nevertheless, several of these are entries from the same *Half-Life* franchise, remaining within the same shooting-and-escape perspective and offering little variety.

In contrast, OpenMind returns a top-5 list in which all five items share the anchor’s horror-and-survival tone while presenting different play styles, and all five still align well with the user’s preferences. For instance, it recommends *F.E.A.R.*, which extends the horror theme toward shooting, and *State of Decay*, which reframes survival as resource management and base-building, while *DARQ* shifts the same dark atmosphere toward puzzle-solving. As a result, OpenMind achieves the best balance between relevance and viewpoint diversity across the entire top-5 list, broadening the user’s horizons and helping to break filter bubbles.

(b) *Quantitative evaluation.* In total, we collected 46 (out of 60) valid responses. In Figure 9(m), ~41.30% of participants confirms that OpenMind achieves the best overall performance. Compared with RAPID (resp. SameTopic), OpenMind achieves 39.13% (resp. 17.40%), 52.17% (resp. 23.91%), 4.35% (resp. 13.05%) and 10.87% (resp. 30.43%) higher Acc, TopCon, ViewD and All, respectively, verifying the effectiveness of balancing both acceptability and diversity in alternative generation. Although Random has higher ViewD than

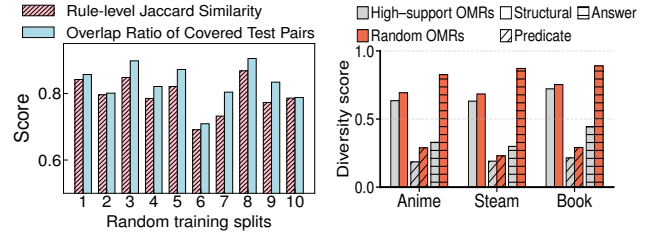


Figure 13: Stability of OMRs

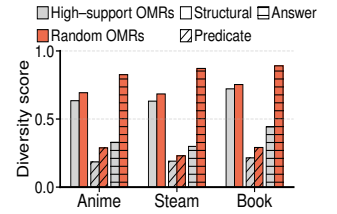


Figure 14: Diversity of OMRs

OpenMind as it samples across-topic items from the entire dataset, its Acc, TopCon and All are lower than OpenMind by 34.79%, 50.00% and 23.91%, respectively, indicating that indiscriminate diversification produces items users are unlikely to accept.

Appendix D-3: Stability of OMRs. On Steam, we tested the stability of OMRs across training samples. As shown in Figure 13, across 10 random training splits, the mined rule sets achieve, on average, 0.7942 rule-level Jaccard similarity (that considers both pattern and predicates in an OMR) and 0.8289 averaged consistency in which test user-item pairs get covered (*i.e.*, the overlap ratio of covered-pair sets across different training splits, not the coverage itself). Thus, OMR discovery is stable across training samples.

Appendix D-4: Diversity of OMRs. We evaluated whether the discovered OMRs Σ_{OMR} capture complementary patterns via three dimensions: (1) *structural diversity*, measured by average pairwise pattern dissimilarity; (2) *predicate diversity*, predicate-set dissimilarity; and (3) *answer diversity*, the average Jaccard distance between covered test pairs. To control rule-set size, we compared two equally sized subsets: $\Sigma_{\text{OMR}}^{\text{sup}}$, containing the highest-support OMRs, and $\Sigma_{\text{OMR}}^{\text{rand}}$, sampled uniformly from Σ_{OMR} . As shown in Figure 14, on average, $\Sigma_{\text{OMR}}^{\text{sup}}$ has 6.79%, 26.36% and 58.72% lower structural, predicate and answer diversity, respectively, than $\Sigma_{\text{OMR}}^{\text{rand}}$ on all datasets, indicating that high-support rules capture dominant patterns. Moreover, Σ_{OMR} covers 6.72% more test pairs than $\Sigma_{\text{OMR}}^{\text{sup}}$, and its coverage gets larger as additional OMRs are included (not shown), showing that lower-support rules contribute complementary rather than

redundant patterns, so Σ_{OMR} as a whole is diverse.